

Bank Churn Prediction

Problem Statement

Context

Businesses like banks which provide service have to worry about problem of 'Customer Churn' i.e. customers leaving and joining another service provider. It is important to understand which aspects of the service influence a customer's decision in this regard. Management can concentrate efforts on improvement of service, keeping in mind these priorities.

Objective

You as a Data scientist with the bank need to build a neural network based classifier that can determine whether a customer will leave the bank or not in the next 6 months.

Data Dictionary

- CustomerId: Unique ID which is assigned to each customer
- Surname: Last name of the customer
- CreditScore: It defines the credit history of the customer.
- Geography: A customer's location
- Gender: It defines the Gender of the customer
- Age: Age of the customer
- Tenure: Number of years for which the customer has been with the bank
- NumOfProducts: refers to the number of products that a customer has purchased through the bank.
- Balance: Account balance
- HasCrCard: It is a categorical variable which decides whether the customer has credit card or not.
- EstimatedSalary: Estimated salary
- isActiveMember: Is is a categorical variable which decides whether the customer is active member of the bank or not (Active member in the sense, using bank products regularly, making transactions etc)
- Exited : whether or not the customer left the bank within six month. It can take two values ** 0=No (Customer did not leave the bank) ** 1=Yes (Customer left the bank)

```
#Installing the libraries with the specified version.
# !pip install tensorflow==2.15.0 scikit-learn==1.2.2 seaborn==0.13.1
    matplotlib==3.7.1 numpy==1.24.1 pandas==1.5.3 -q --user --no-
    warn-script-location
```

Importing necessary libraries

```
# Libraries to help with reading and manipulating data
import pandas as pd
```

```
import numpy as np

# libraries to help with data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Library to split data
from sklearn.model_selection import train_test_split

# library to import to standardize the data
from sklearn.preprocessing import StandardScaler, LabelEncoder

# importing different functions to build models
import tensorflow as tf
from tensorflow import keras
from keras import backend
from keras.models import Sequential
from keras.layers import Dense, Dropout

# importing SMOTE
from imblearn.over_sampling import SMOTE

# importing metrics
from sklearn.metrics import
    confusion_matrix, roc_curve, classification_report, recall_score

import random

# Library to avoid the warnings
import warnings
warnings.filterwarnings("ignore")

# Set the seed using keras.utils.set_random_seed. This will set:
# 1) `numpy` seed
# 2) backend random seed
# 3) `python` random seed
keras.utils.set_random_seed(812)

# If using TensorFlow, this will make GPU ops as deterministic as
    possible,
# but it will affect the overall performance, so be mindful of that.
tf.config.experimental.enable_op_determinism()
```

Loading the dataset

```
from google.colab import drive
drive.mount('/content/drive')
file = r'/content/drive/MyDrive/AI Class/Projects/Project 4/bank-1.csv'
data = pd.read_csv(file)

Mounted at /content/drive
```

Data Overview

```
data.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balan
0	1	15634602	Hargrave	619	France	Female	42	2	0.00
1	2	15647311	Hill	608	Spain	Female	41	1	83807.4

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance
2	3	15619304	Onio	502	France	Female	42	8	159660
3	4	15701354	Boni	699	France	Female	39	1	0.00
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510

```
# checking shape of the data
print(f"There are {data.shape[0]} rows and {data.shape[1]} columns.")
```

There are 10000 rows and 14 columns.

```
# let's view a sample of the data
data.sample(n=10, random_state=1)
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure
9953	9954	15655952	Burke	550	France	Male	47	2
3850	3851	15775293	Stephenson	680	France	Male	34	3
4962	4963	15665088	Gordon	531	France	Female	42	2
3886	3887	15720941	Tien	710	Germany	Male	34	8
5437	5438	15733476	Gonzalez	543	Germany	Male	30	6
8517	8518	15671800	Robinson	688	France	Male	20	8
2041	2042	15709846	Yeh	840	France	Female	39	1
1989	1990	15622454	Zaitsev	695	Spain	Male	28	0
1933	1934	15815560	Bogle	666	Germany	Male	74	7
9984	9985	15696175	Echezonachukwu	602	Germany	Male	35	7

```
# let's create a copy of the data to avoid any changes to original
data
```

```
df = data.copy()
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   RowNumber             10000 non-null  int64
 1   CustomerId            10000 non-null  int64
 2   Surname               10000 non-null  object
 3   CreditScore           10000 non-null  int64
 4   Geography             10000 non-null  object
 5   Gender                10000 non-null  object
 6   Age                   10000 non-null  int64
 7   Tenure                10000 non-null  int64
 8   Balance               10000 non-null  float64
 9   NumOfProducts         10000 non-null  int64
10   HasCrCard             10000 non-null  int64
11   IsActiveMember        10000 non-null  int64
12   EstimatedSalary       10000 non-null  float64
13   Exited                10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

```
# checking for duplicate values
df.duplicated().sum()
```

```
np.int64(0)

df[df.duplicated(keep=False) == True]
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance
--	-----------	------------	---------	-------------	-----------	--------	-----	--------	---------

```
df.isnull().sum()
```

	0								
RowNumber	0								
CustomerId	0								
Surname	0								
CreditScore	0								
Geography	0								
Gender	0								
Age	0								
Tenure	0								
Balance	0								
NumOfProducts	0								
HasCrCard	0								
IsActiveMember	0								
EstimatedSalary	0								
Exited	0								

```
dtype: int64

df.nunique()
```

	0								
RowNumber	10000								
CustomerId	10000								
Surname	2932								
CreditScore	460								
Geography	3								
Gender	2								
Age	70								
Tenure	11								
Balance	6382								
NumOfProducts	4								
HasCrCard	2								
IsActiveMember	2								
EstimatedSalary	9999								
Exited	2								

```
dtype: int64

df = df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1)
```


Exploratory Data Analysis

Univariate Analysis

```
# creating a copy of the dataframe
df1 = df.copy()

# function to create labeled barplots

def labeled_barplot(data, feature, perc=False, n=None):
    """
    Barplot with percentage at the top

    data: dataframe
    feature: dataframe column
    perc: whether to display percentages instead of count (default is
          False)
    n: displays the top n category levels (default is None, i.e.,
        display all levels)
    """

    total = len(data[feature]) # length of the column
    count = data[feature].nunique()
    if n is None:
        plt.figure(figsize=(count + 1, 5))
    else:
        plt.figure(figsize=(n + 1, 5))

    plt.xticks(rotation=90, fontsize=15)
    ax = sns.countplot(
        data=data,
        x=feature,
        palette="Paired",
        order=data[feature].value_counts().index[:n].sort_values(),
    )

    for p in ax.patches:
        if perc == True:
            label = "{:.1f}%".format(
                100 * p.get_height() / total
            ) # percentage of each class of the category
        else:
            label = p.get_height() # count of each level of the
            category

        x = p.get_x() + p.get_width() / 2 # width of the plot
        y = p.get_height() # height of the plot

        ax.annotate(
            label,
            (x, y),
            ha="center",
            va="center",
            size=12,
            xytext=(0, 5),
            textcoords="offset points",
        ) # annotate the percentage

    plt.show() # show the plot
```

```
# function to plot a boxplot and a histogram along the same scale.
```

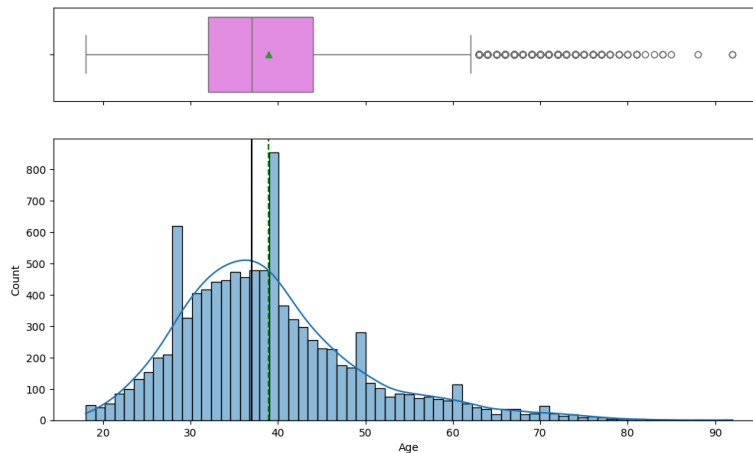
```
def histogram_boxplot(data, feature, figsize=(12, 7), kde=False,
                      bins=None):
    """
    Boxplot and histogram combined

    data: dataframe
    feature: dataframe column
    figsize: size of figure (default (12,7))
    kde: whether to the show density curve (default False)
    bins: number of bins for histogram (default None)
    """

    f2, (ax_box2, ax_hist2) = plt.subplots(
        nrows=2, # Number of rows of the subplot grid= 2
        sharex=True, # x-axis will be shared among all subplots
        gridspec_kw={"height_ratios": (0.25, 0.75)},
        figsize=figsize,
    ) # creating the 2 subplots
    sns.boxplot(
        data=data, x=feature, ax=ax_box2, showmeans=True,
        color="violet"
    ) # boxplot will be created and a star will indicate the mean
        value of the column
    sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2, bins=bins,
        palette="winter"
    ) if bins else sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2
    ) # For histogram
    ax_hist2.axvline(
        data[feature].mean(), color="green", linestyle="--"
    ) # Add mean to the histogram
    ax_hist2.axvline(
        data[feature].median(), color="black", linestyle="-"
    ) # Add median to the histogram
```

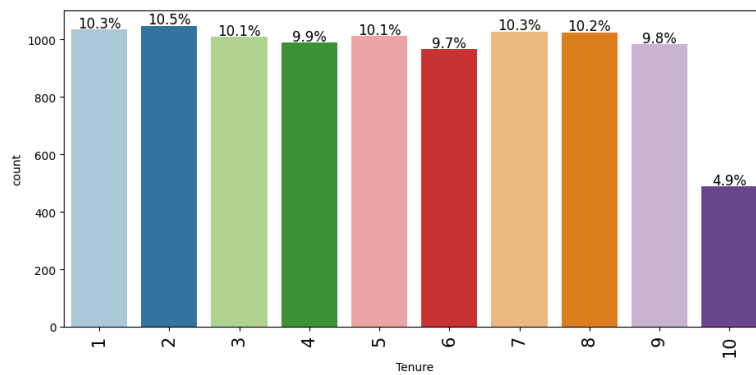
```
histogram_boxplot(df1, "Age", kde=True)
```

```
#STD of age is in the middle, late 30s to early 40s
```



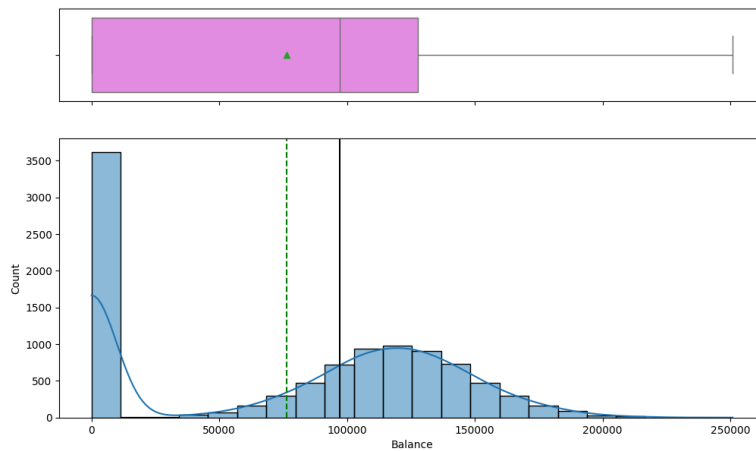
```
labeled_barplot(df1, "Tenure", perc=True, n=10)
```

```
#Highest tenure of members is 10 yrs but also the lowest count,
highest count is second yr members with post 1 yr usage
```



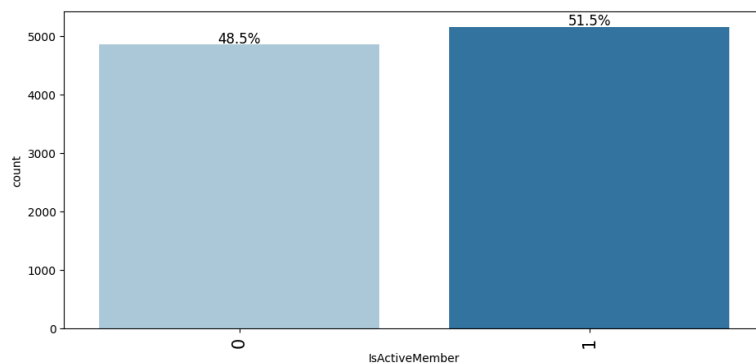
```
histogram_boxplot(df1, "Balance", kde=True)
```

#majority of people dont have anything in their accs as of yet since they might be new, median is nearing \$125k



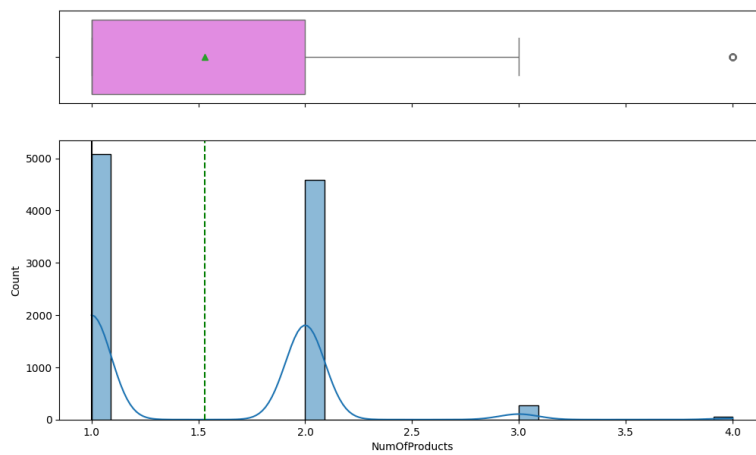
```
labeled_barplot(df1, "IsActiveMember", perc=True, n=10)
```

#fairly even split for active vs non active members

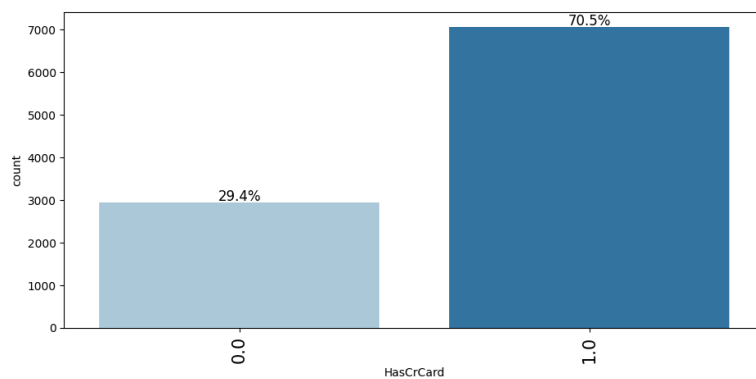


```
histogram_boxplot(df1, "NumOfProducts", kde=True)
```

#members either have 1 product, which is the highest count, 2 which is second or 3, the least, fairly easy graph to understand

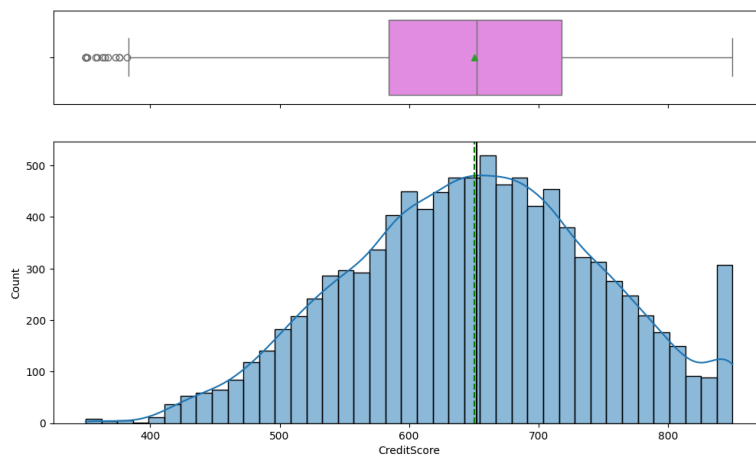


```
labeled_barplot(df1, "HasCrCard", perc=True, n=10)
# yes or no, 70% of members have a card, good area to avoid, they wont
# leave, 30% is target
```



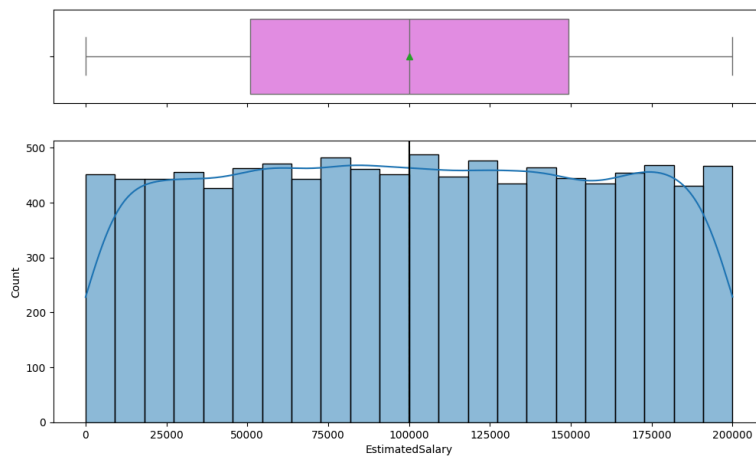
```
histogram_boxplot(df1, "CreditScore", kde=True)
```

```
# some outliers in the front, STD is leaning near 650 credit score,
# not too bad but nothing great
```



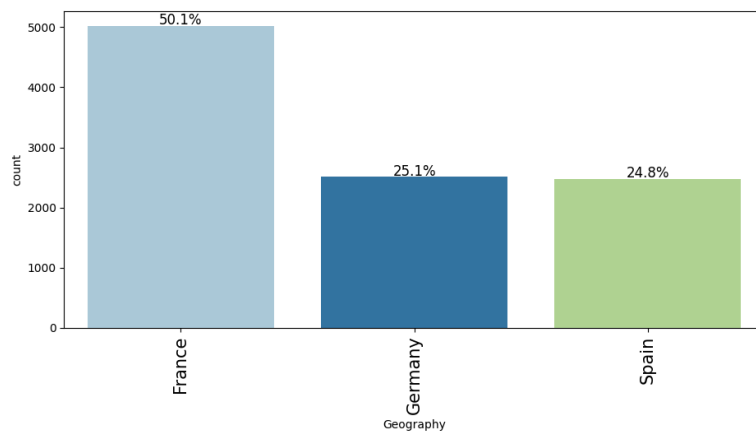
```
histogram_boxplot(df1, "EstimatedSalary", kde=True)
```

```
# hard to deduct anything from this graph, people make money, or dont
```



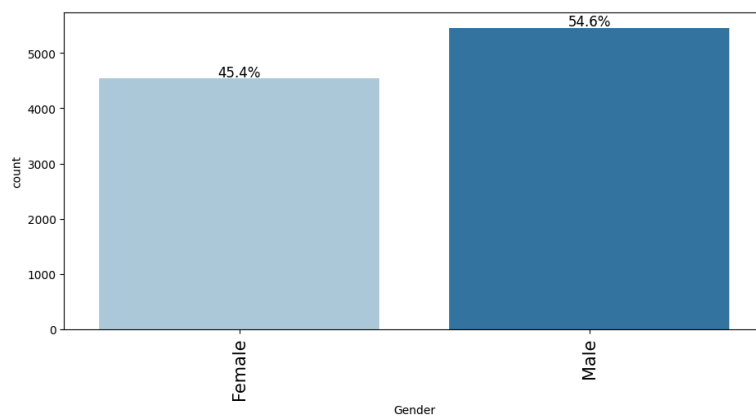
```
labeled_barplot(df, "Geography", perc=True, n=10)
```

```
# breakdown of locations of members, good idea to look into churn rate  
# of those folks in these regions
```



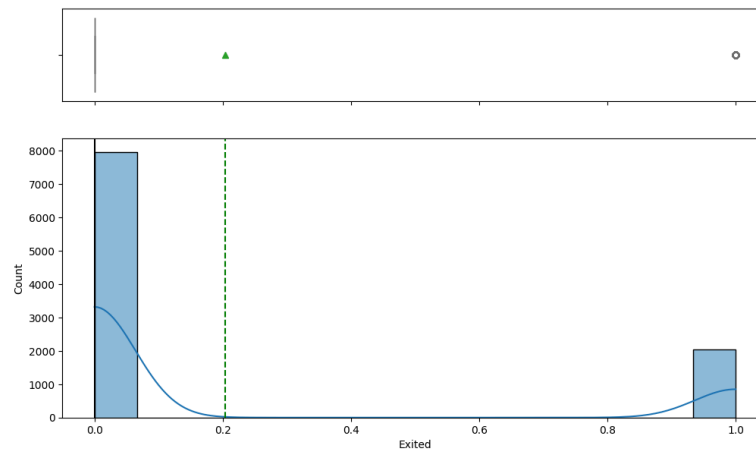
```
labeled_barplot(df, "Gender", perc=True, n=10)
```

```
# gender graph
```



```
histogram_boxplot(df1, "Exited", kde=True)
```

```
# left or stayed
```



Bivariate Analysis

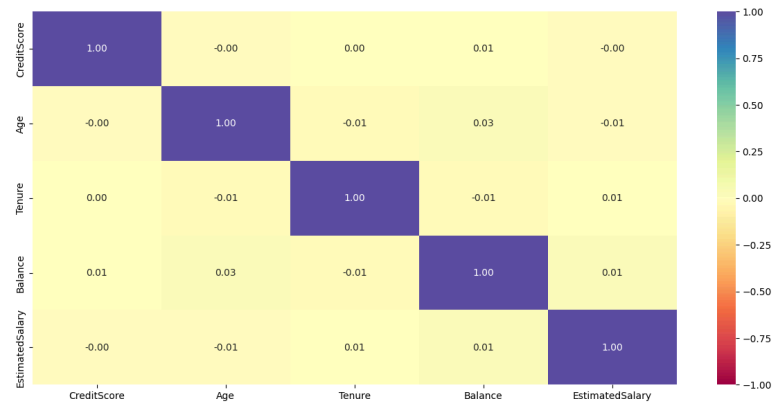
function to plot stacked bar chart

```
def stacked_barplot(data, predictor, target):
    """
    Print the category counts and plot a stacked bar chart

    data: dataframe
    predictor: independent variable
    target: target variable
    """
    count = data[predictor].nunique()
    sorter = data[target].value_counts().index[-1]
    tab1 = pd.crosstab(data[predictor], data[target],
                       margins=True).sort_values(
        by=sorter, ascending=False
    )
    print(tab1)
    print("-" * 120)
    tab = pd.crosstab(data[predictor], data[target],
                      normalize="index").sort_values(
        by=sorter, ascending=False
    )
    tab.plot(kind="bar", stacked=True, figsize=(count + 1, 5))
    plt.legend(
        loc="lower left",
        frameon=False,
    )
    plt.legend(loc="upper left", bbox_to_anchor=(1, 1))
    plt.show()

# defining the list of numerical columns
cols_list = ["CreditScore", "Age", "Tenure", "Balance", "EstimatedSalary"]

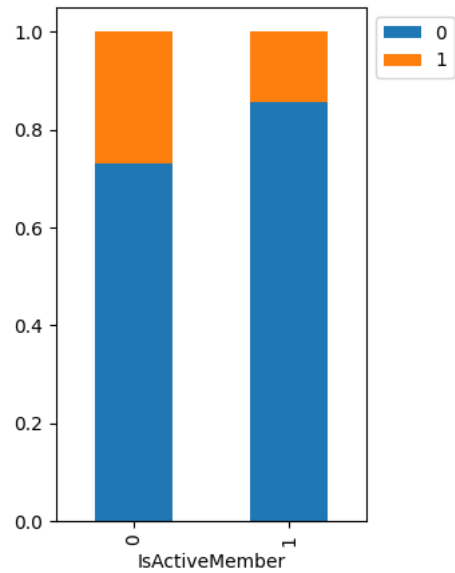
plt.figure(figsize=(15, 7))
sns.heatmap(df1[cols_list].corr(), annot=True, vmin=-1, vmax=1,
            fmt=".2f", cmap="spectral")
plt.show()
```



```
stacked_barplot(df1, "IsActiveMember", "Exited")
```

```
# non active members leaving is usual
```

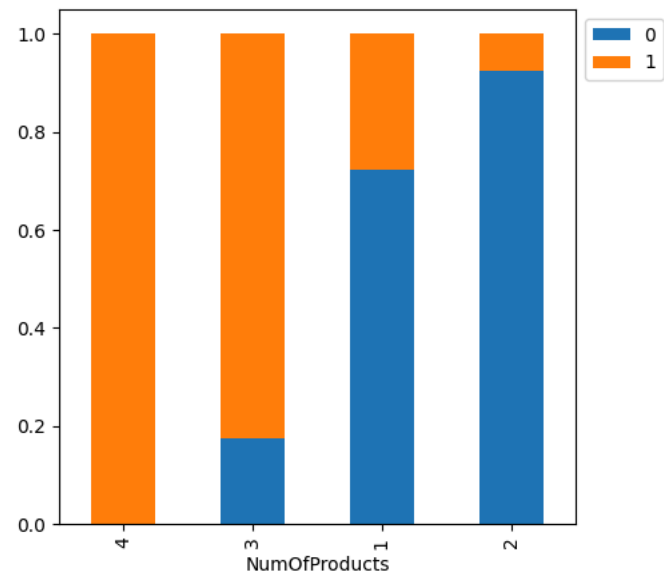
Exited	0	1	All
IsActiveMember			
All	7963	2037	10000
0	3547	1302	4849
1	4416	735	5151



```
stacked_barplot(df1, "NumOfProducts", "Exited")
```

```
# higher the products the higher likelihood they leave, fewer stay, good correlation
```

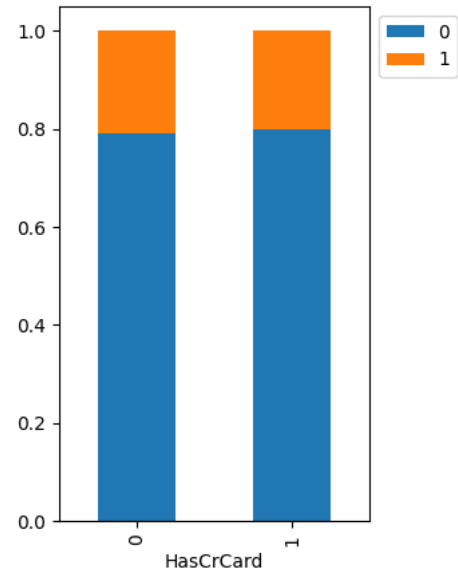
Exited	0	1	All
NumOfProducts			
All	7963	2037	10000
1	3675	1409	5084
2	4242	348	4590
3	46	220	266
4	0	60	60



```
stacked_barplot(df1, "HasCrCard", "Exited")
```

even down the line

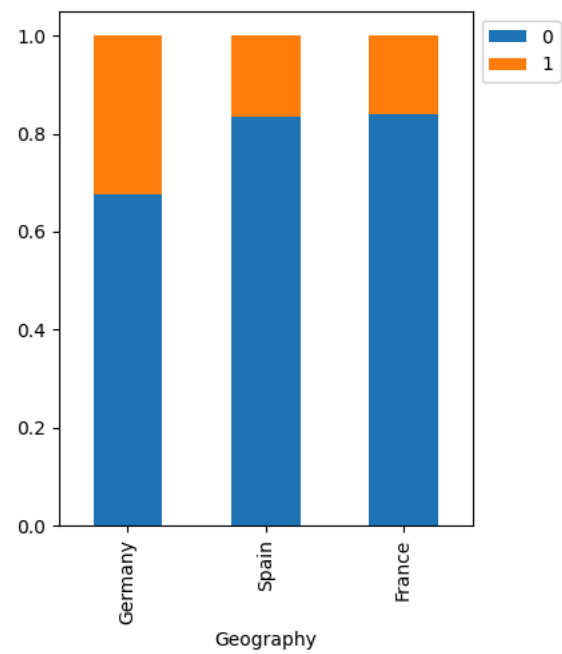
Exited	0	1	All
HasCrCard			
All	7963	2037	10000
1	5631	1424	7055
0	2332	613	2945



```
stacked_barplot(df, "Geography", "Exited")
```

higher numbers of folks left in germany, might be something to look into, also opposite for the other 2 regions

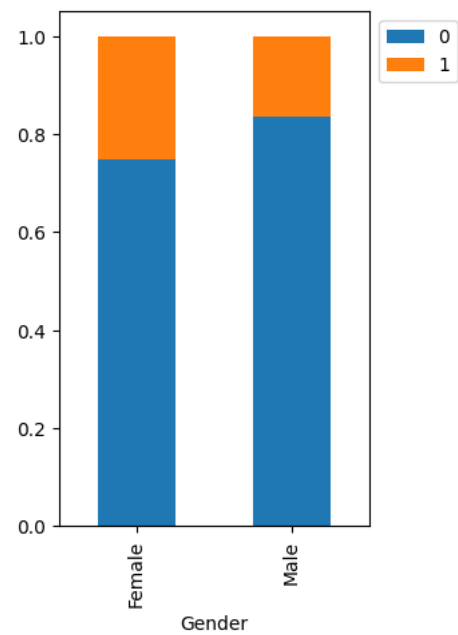
Exited	0	1	All
Geography			
All	7963	2037	10000
Germany	1695	814	2509
France	4204	810	5014
Spain	2064	413	2477



```
stacked_barplot(df, "Gender", "Exited")
```

higher number of females leaving than males

Exited	0	1	All
Gender			
All	7963	2037	10000
Female	3404	1139	4543
Male	4559	898	5457



```
stacked_barplot(df, "Age", "Exited")
```

as age increases, likelihood of member leaving slims down, too much hassle, younger people are making changes

Exited	0	1	All
Age			
All	7963	2037	10000
46	135	91	226
40	343	89	432
43	209	88	297
45	142	87	229
..	...	...	...
81	4	0	4
83	1	0	1
85	1	0	1
88	1	0	1
92	2	0	2

[71 rows x 3 columns]



```
stacked_barplot(df, "Balance", "Exited")
```

```
# too big of a graph to deduce anything, but after some point its all
# blue meaning people are staying
```

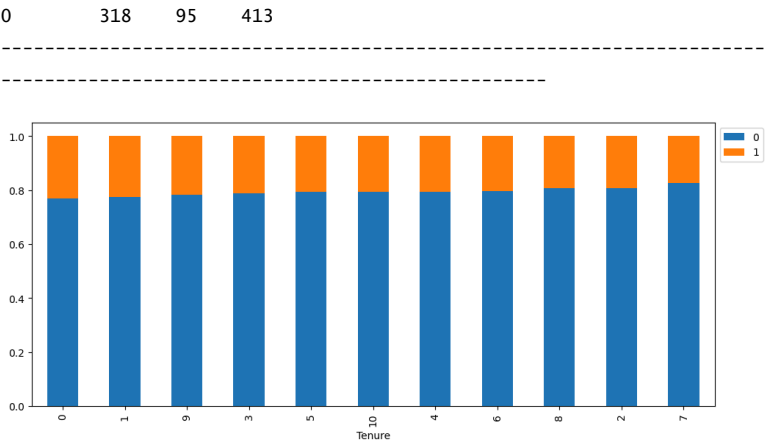
Exited	0	1	All
Balance			
All	7963	2037	10000
0.0	3117	500	3617
117206.3	0	1	1
91592.06	0	1	1
91536.93	0	1	1
...	...	...	...
212778.2	1	0	1
212696.32	1	0	1
212692.97	1	0	1
212314.03	1	0	1
206868.78	1	0	1

[6383 rows x 3 columns]

```
stacked_barplot(df, "Tenure", "Exited")
```

```
# people with no tenure, < a year, are leaving sooner than later
```

Exited	0	1	All
Tenure			
All	7963	2037	10000
1	803	232	1035
9	771	213	984
3	796	213	1009
5	803	209	1012
4	786	203	989
2	847	201	1048
8	828	197	1025
6	771	196	967
7	851	177	1028
10	389	101	490



```
stacked_barplot(df1, "Creditscore", "Exited")

# as credit score increases, people are more likely to stay
```

Exited	0	1	All
Creditscore			
All	7963	2037	10000
850	190	43	233
651	33	17	50
705	37	16	53
637	32	14	46
...	...	...	...
830	2	0	2
801	7	0	7
837	8	0	8
810	6	0	6
848	5	0	5

[461 rows x 3 columns]

```
stacked_barplot(df1, "EstimatedSalary", "Exited")
```

Exited	0	1	All
EstimatedSalary			
All	7963	2037	10000
91098.76	0	1	1
91167.19	0	1	1
91556.57	0	1	1
91560.63	0	1	1
...	...	...	...
90304.01	1	0	1
90284.47	1	0	1
90280.7	1	0	1
90273.85	1	0	1
90605.05	1	0	1

[10000 rows x 3 columns]

Data Preprocessing

Dummy Variable Creation

```
df1 = pd.get_dummies(df1,columns=df1.select_dtypes(include=
["object"]).columns.tolist(),drop_first=True)
df1 = df1.astype(float)
df1.head()
```

#creating dummy vars of type objects

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
0	619.0	42.0	2.0	0.00	1.0	1.0	1.0	101348.5
1	608.0	41.0	1.0	83807.86	1.0	0.0	1.0	112542.0
2	502.0	42.0	8.0	159660.80	3.0	1.0	0.0	113931.0
3	699.0	39.0	1.0	0.00	2.0	0.0	0.0	93826.6
4	850.0	43.0	2.0	125510.82	1.0	1.0	1.0	79084.1

Train-validation-test Split

```
x = df1.drop(['Exited'],axis=1) # Credit Score through Estimated
Salary
y = df1['Exited'] # Exited

# Splitting the dataset into the Training and Testing set.

X_large, X_test, y_large, y_test = train_test_split(X, y, test_size =
0.3, random_state = 42,stratify=y,shuffle = True) ## Complete
the code to Split the X and y and obtain test set

# Splitting the dataset into the Training and Testing set.

X_train, X_val, y_train, y_val = train_test_split(X_large, y_large,
test_size = 0.5, random_state = 42,stratify=y_large, shuffle
= True) ## complete the code to Split X_large and y_large to
obtain train and validation sets

print(X_train.shape, X_val.shape, X_test.shape)

(3500, 11) (3500, 11) (3000, 11)

print(y_train.shape, y_val.shape, y_test.shape)

(3500,) (3500,) (3000,)

# # # Encoding the categorical variables using one-hot encoding
# x = pd.get_dummies(
#     x,
#     columns=["Geography","Gender"],
#     drop_first=True,
# )

# x = x.astype(float)

#kept getting an issue trying to encode the cols, something with not
allowing gender and geography to run, had the code but left
it out
```

Data Normalization

```
# creating an instance of the standard scaler
sc = StandardScaler()
```

```
cols_list = X_train.select_dtypes(include=['float64',
                                           'int64']).columns.tolist()

X_train[cols_list] = sc.fit_transform(X_train[cols_list])
X_val[cols_list] = sc.transform(X_val[cols_list])    ## Complete the
code to specify the columns to normalize
X_test[cols_list] = sc.transform(X_test[cols_list])  ## Complete the
code to specify the columns to normalize
```

Model Building

Model Evaluation Criterion

Write down the logic for choosing the metric that would be the best metric for this business scenario.

-

```
def plot(history, name):
    """
    Function to plot loss/accuracy

    history: an object which stores the metrics and losses.
    name: can be one of Loss or Accuracy
    """

    fig, ax = plt.subplots() #Creating a subplot with figure and axes.
    plt.plot(history.history[name]) #Plotting the train accuracy or
    train loss
    plt.plot(history.history['val_'+name]) #Plotting the validation
    accuracy or validation loss

    plt.title('Model ' + name.capitalize()) #Defining the title of the
    plot.
    plt.ylabel(name.capitalize()) #Capitalizing the first letter.
    plt.xlabel('Epoch') #Defining the label for the x-axis.
    fig.legend(['Train', 'Validation'], loc="outside right upper")
    #Defining the legend, loc controls the position of the
    legend.

# defining a function to compute different metrics to check
performance of a classification model built using statsmodels
def model_performance_classification(
    model, predictors, target, threshold=0.5
):
    """
    Function to compute different metrics to check classification
    model performance

    model: classifier
    predictors: independent variables
    target: dependent variable
    threshold: threshold for classifying the observation as class 1
    """

    pred = model.predict(predictors) > threshold

    acc = accuracy_score(target, pred) # to compute Accuracy
    recall = recall_score(target, pred, average='weighted') # to
    compute Recall
    precision = precision_score(target, pred, average='weighted') #
    to compute Precision
    f1 = f1_score(target, pred, average='weighted') # to compute F1-
    score

    # creating a dataframe of metrics
    df_perf = pd.DataFrame(
        {"Accuracy": acc, "Recall": recall, "Precision": precision,
        "F1 Score": f1},
```

```

        index=[0],
    )

    return df_perf

def make_confusion_matrix(actual_targets, predicted_targets):
    """
    To plot the confusion_matrix with percentages

    actual_targets: actual target (dependent) variable values
    predicted_targets: predicted target (dependent) variable values
    """
    cm = confusion_matrix(actual_targets, predicted_targets)
    labels = np.asarray(
        [
            ["{0:0.0f}".format(item) + "\n{0:.2%}".format(item /
            cm.flatten().sum())]
            for item in cm.flatten()
        ]
    ).reshape(cm.shape[0], cm.shape[1])

    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=labels, fmt="")
    plt.ylabel("True label")
    plt.xlabel("Predicted label")

train_metric_df = pd.DataFrame(columns=["recall"])
valid_metric_df = pd.DataFrame(columns=["recall"])

# Calculate class weights for imbalanced dataset
cw = (y_train.shape[0]) / np.bincount(y_train)

# Create a dictionary mapping class indices to their respective class
weights
cw_dict = {}
for i in range(cw.shape[0]):
    cw_dict[i] = cw[i]

cw_dict

{0: np.float64(1.2558306422676713), 1: np.float64(4.908835904628331)}

# defining the batch size and # epochs upfront as we'll be using the
same values for all models
epochs = 25
batch_size = 64

```

Neural Network with SGD Optimizer

```

backend.clear_session()
#Fixing the seed for random number generators so that we can ensure we
receive the same output everytime
np.random.seed(2)
random.seed(2)
tf.random.set_seed(2)

#Initializing the neural network
model_0 = Sequential()
# Adding the input layer with 64 neurons and relu as activation
function
model_0.add(Dense(64, activation='relu', input_dim =
X_train.shape[1]))
# Complete the code to add a hidden layer (specify the # of neurons
and the activation function)
model_0.add(Dense(7,activation="relu"))

```

```
# Complete the code to add the output layer with the number of neurons
required.
```

```
model_0.add(Dense(1,activation="sigmoid"))
```

```
model_0.summary()
```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 64)	
dense_1 (Dense)	(None, 7)	
dense_2 (Dense)	(None, 1)	



Total params: 1,231 (4.81 KB)

Trainable params: 1,231 (4.81 KB)

Non-trainable params: 0 (0.00 B)

```
optimizer = tf.keras.optimizers.SGD(0.001) # defining SGD as the
optimizer to be used
```

```
metric = keras.metrics.Recall()
```

```
model_0.compile(loss='binary_crossentropy',optimizer=optimizer,metrics=
[metric])
```

```
# Fitting the ANN
```

```
history_0 = model_0.fit(
    x_train, y_train,
    batch_size=25,    ## Complete the code to specify the batch size
                     to use
    validation_data=(x_val,y_val),
    epochs=55,    ## Complete the code to specify the number of epochs
    verbose=1
)
```

Epoch 1/55

```
140/140 _____ 1s 7ms/step - loss: 0.4554 - recall:
0.0000e+00 - val_loss: 0.4600 - val_recall: 0.0000e+00
```

Epoch 2/55

```
140/140 _____ 1s 4ms/step - loss: 0.4547 - recall:
0.0000e+00 - val_loss: 0.4592 - val_recall: 0.0000e+00
```

Epoch 3/55

```
140/140 _____ 1s 4ms/step - loss: 0.4539 - recall:
0.0000e+00 - val_loss: 0.4585 - val_recall: 0.0000e+00
```

Epoch 4/55

```
140/140 _____ 1s 4ms/step - loss: 0.4532 - recall:
0.0000e+00 - val_loss: 0.4578 - val_recall: 0.0000e+00
```

Epoch 5/55

```
140/140 _____ 1s 4ms/step - loss: 0.4525 - recall:
0.0000e+00 - val_loss: 0.4571 - val_recall: 0.0000e+00
```

Epoch 6/55

```
140/140 _____ 1s 5ms/step - loss: 0.4518 - recall:
0.0000e+00 - val_loss: 0.4564 - val_recall: 0.0000e+00
```

Epoch 7/55

```
140/140 _____ 1s 5ms/step - loss: 0.4511 - recall:
0.0000e+00 - val_loss: 0.4557 - val_recall: 0.0000e+00
```

Epoch 8/55

```
140/140 _____ 2s 8ms/step - loss: 0.4505 - recall:
0.0000e+00 - val_loss: 0.4551 - val_recall: 0.0000e+00
```

Epoch 9/55

```
140/140 _____ 1s 8ms/step - loss: 0.4498 - recall:
0.0000e+00 - val_loss: 0.4544 - val_recall: 0.0000e+00
```

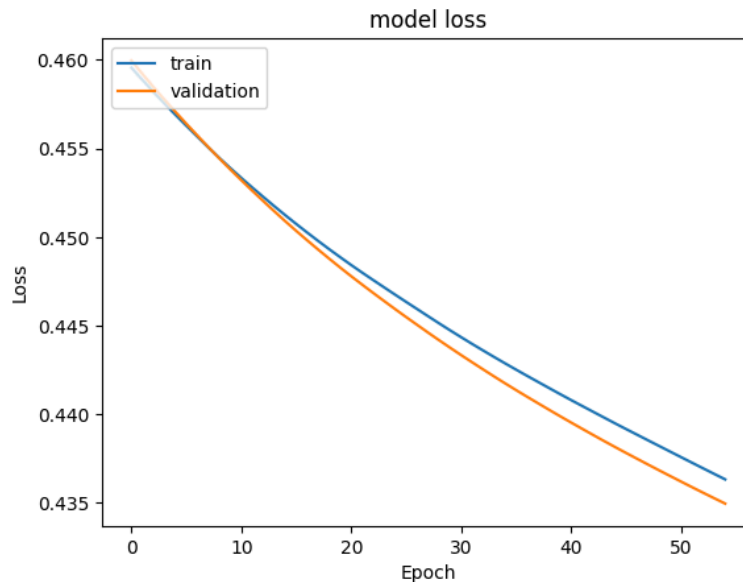
Epoch 10/55
140/140 ————— 1s 7ms/step - loss: 0.4492 - recall:
0.0000e+00 - val_loss: 0.4538 - val_recall: 0.0000e+00
Epoch 11/55
140/140 ————— 2s 12ms/step - loss: 0.4486 - recall:
0.0000e+00 - val_loss: 0.4532 - val_recall: 0.0000e+00
Epoch 12/55
140/140 ————— 2s 11ms/step - loss: 0.4479 - recall:
0.0000e+00 - val_loss: 0.4526 - val_recall: 0.0000e+00
Epoch 13/55
140/140 ————— 2s 5ms/step - loss: 0.4473 - recall:
0.0000e+00 - val_loss: 0.4520 - val_recall: 0.0000e+00
Epoch 14/55
140/140 ————— 1s 4ms/step - loss: 0.4468 - recall:
0.0000e+00 - val_loss: 0.4515 - val_recall: 0.0000e+00
Epoch 15/55
140/140 ————— 1s 4ms/step - loss: 0.4462 - recall:
0.0000e+00 - val_loss: 0.4509 - val_recall: 0.0000e+00
Epoch 16/55
140/140 ————— 1s 4ms/step - loss: 0.4456 - recall:
0.0000e+00 - val_loss: 0.4503 - val_recall: 0.0000e+00
Epoch 17/55
140/140 ————— 1s 4ms/step - loss: 0.4451 - recall:
0.0000e+00 - val_loss: 0.4498 - val_recall: 0.0000e+00
Epoch 18/55
140/140 ————— 1s 4ms/step - loss: 0.4445 - recall:
0.0000e+00 - val_loss: 0.4493 - val_recall: 0.0000e+00
Epoch 19/55
140/140 ————— 1s 4ms/step - loss: 0.4440 - recall:
0.0000e+00 - val_loss: 0.4488 - val_recall: 0.0000e+00
Epoch 20/55
140/140 ————— 1s 4ms/step - loss: 0.4435 - recall:
0.0000e+00 - val_loss: 0.4483 - val_recall: 0.0000e+00
Epoch 21/55
140/140 ————— 1s 4ms/step - loss: 0.4429 - recall:
0.0000e+00 - val_loss: 0.4478 - val_recall: 0.0014
Epoch 22/55
140/140 ————— 2s 7ms/step - loss: 0.4425 - recall:
0.0000e+00 - val_loss: 0.4473 - val_recall: 0.0014
Epoch 23/55
140/140 ————— 1s 7ms/step - loss: 0.4420 - recall:
0.0000e+00 - val_loss: 0.4468 - val_recall: 0.0014
Epoch 24/55
140/140 ————— 1s 8ms/step - loss: 0.4415 - recall:
0.0000e+00 - val_loss: 0.4464 - val_recall: 0.0014
Epoch 25/55
140/140 ————— 1s 8ms/step - loss: 0.4410 - recall:
0.0000e+00 - val_loss: 0.4459 - val_recall: 0.0014
Epoch 26/55
140/140 ————— 1s 5ms/step - loss: 0.4406 - recall:
0.0000e+00 - val_loss: 0.4455 - val_recall: 0.0014
Epoch 27/55
140/140 ————— 1s 5ms/step - loss: 0.4401 - recall:
0.0000e+00 - val_loss: 0.4450 - val_recall: 0.0014
Epoch 28/55
140/140 ————— 1s 4ms/step - loss: 0.4396 - recall: 0.0012
- val_loss: 0.4446 - val_recall: 0.0014
Epoch 29/55
140/140 ————— 1s 4ms/step - loss: 0.4392 - recall: 0.0012
- val_loss: 0.4442 - val_recall: 0.0014
Epoch 30/55

140/140 ————— 1s 4ms/step - loss: 0.4387 - recall: 0.0012
- val_loss: 0.4438 - val_recall: 0.0014
Epoch 31/55
140/140 ————— 1s 4ms/step - loss: 0.4383 - recall: 0.0012
- val_loss: 0.4433 - val_recall: 0.0014
Epoch 32/55
140/140 ————— 1s 4ms/step - loss: 0.4378 - recall: 0.0012
- val_loss: 0.4429 - val_recall: 0.0014
Epoch 33/55
140/140 ————— 1s 4ms/step - loss: 0.4374 - recall: 0.0012
- val_loss: 0.4425 - val_recall: 0.0014
Epoch 34/55
140/140 ————— 1s 4ms/step - loss: 0.4370 - recall: 0.0012
- val_loss: 0.4421 - val_recall: 0.0014
Epoch 35/55
140/140 ————— 1s 4ms/step - loss: 0.4366 - recall: 0.0012
- val_loss: 0.4417 - val_recall: 0.0014
Epoch 36/55
140/140 ————— 1s 4ms/step - loss: 0.4361 - recall: 0.0012
- val_loss: 0.4414 - val_recall: 0.0014
Epoch 37/55
140/140 ————— 2s 8ms/step - loss: 0.4357 - recall: 0.0012
- val_loss: 0.4410 - val_recall: 0.0014
Epoch 38/55
140/140 ————— 1s 6ms/step - loss: 0.4354 - recall: 0.0012
- val_loss: 0.4406 - val_recall: 0.0014
Epoch 39/55
140/140 ————— 1s 6ms/step - loss: 0.4350 - recall: 0.0012
- val_loss: 0.4403 - val_recall: 0.0014
Epoch 40/55
140/140 ————— 1s 5ms/step - loss: 0.4346 - recall: 0.0012
- val_loss: 0.4399 - val_recall: 0.0014
Epoch 41/55
140/140 ————— 1s 5ms/step - loss: 0.4342 - recall: 0.0012
- val_loss: 0.4395 - val_recall: 0.0014
Epoch 42/55
140/140 ————— 1s 4ms/step - loss: 0.4338 - recall: 0.0012
- val_loss: 0.4392 - val_recall: 0.0014
Epoch 43/55
140/140 ————— 1s 4ms/step - loss: 0.4334 - recall: 0.0012
- val_loss: 0.4388 - val_recall: 0.0042
Epoch 44/55
140/140 ————— 1s 4ms/step - loss: 0.4331 - recall: 0.0035
- val_loss: 0.4385 - val_recall: 0.0056
Epoch 45/55
140/140 ————— 1s 4ms/step - loss: 0.4327 - recall: 0.0049
- val_loss: 0.4382 - val_recall: 0.0056
Epoch 46/55
140/140 ————— 1s 4ms/step - loss: 0.4323 - recall: 0.0066
- val_loss: 0.4378 - val_recall: 0.0070
Epoch 47/55
140/140 ————— 1s 4ms/step - loss: 0.4320 - recall: 0.0066
- val_loss: 0.4375 - val_recall: 0.0084
Epoch 48/55
140/140 ————— 1s 4ms/step - loss: 0.4316 - recall: 0.0084
- val_loss: 0.4372 - val_recall: 0.0112
Epoch 49/55
140/140 ————— 1s 4ms/step - loss: 0.4312 - recall: 0.0084
- val_loss: 0.4368 - val_recall: 0.0126
Epoch 50/55
140/140 ————— 1s 4ms/step - loss: 0.4309 - recall: 0.0092

```
- val_loss: 0.4365 - val_recall: 0.0140
Epoch 51/55
140/140 ————— 1s 5ms/step - loss: 0.4305 - recall: 0.0113
- val_loss: 0.4362 - val_recall: 0.0154
Epoch 52/55
140/140 ————— 1s 8ms/step - loss: 0.4302 - recall: 0.0113
- val_loss: 0.4359 - val_recall: 0.0154
Epoch 53/55
140/140 ————— 1s 8ms/step - loss: 0.4298 - recall: 0.0131
- val_loss: 0.4356 - val_recall: 0.0182
Epoch 54/55
140/140 ————— 1s 7ms/step - loss: 0.4294 - recall: 0.0142
- val_loss: 0.4353 - val_recall: 0.0210
Epoch 55/55
140/140 ————— 1s 4ms/step - loss: 0.4291 - recall: 0.0142
- val_loss: 0.4350 - val_recall: 0.0224
```

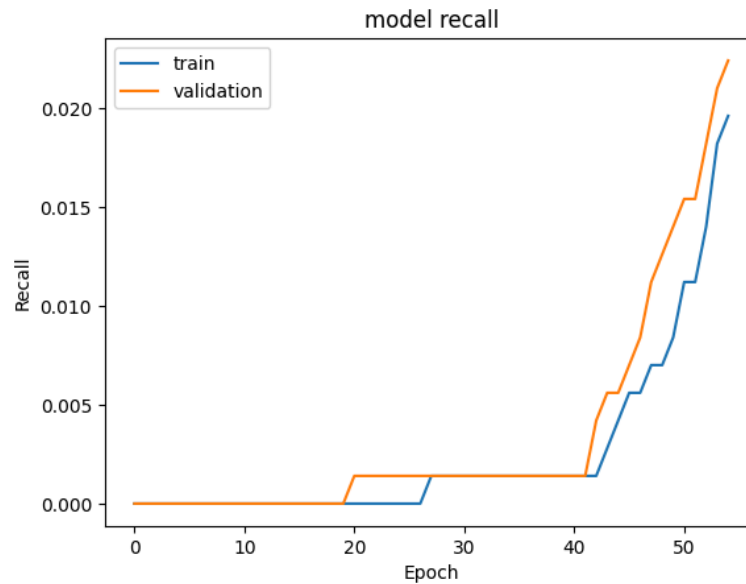
```
#Plotting Train Loss vs Validation Loss
plt.plot(history_0.history['loss'])
plt.plot(history_0.history['val_loss'])
plt.title('model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```

```
# model is learning slowly, low overfitting, some room for improvement
but first model did good
```



```
#Plotting Train recall vs Validation recall
# print(history_0.history.keys())
plt.plot(history_0.history['recall'])
plt.plot(history_0.history['val_recall'])
plt.title('model recall')
plt.ylabel('Recall')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```

```
# takes the model a few epochs to do something, wasnt learning
something before but picked up at 40, data might be
imbalanced
```



```
#Predicting the results using best as a threshold
```

```
y_train_pred = model_0.predict(X_train)
```

```
y_train_pred = (y_train_pred > 0.5)
```

```
y_train_pred
```

```
110/110 ————— 0s 3ms/step
```

```
array([[False],
       [False],
       [False],
       ...,
       [False],
       [False],
       [False]])
```

```
#Predicting the results using best as a threshold
```

```
y_val_pred = model_0.predict(X_val) ## Complete the code to make
prediction on the validation set
```

```
y_val_pred = (y_val_pred > 0.5)
```

```
y_val_pred
```

```
110/110 ————— 0s 2ms/step
```

```
array([[False],
       [False],
       [False],
       ...,
       [False],
       [False],
       [False]])
```

```
model_name = "NN with SGD"
```

```
train_metric_df.loc[model_name] = recall_score(y_train, y_train_pred)
```

```
valid_metric_df.loc[model_name] = recall_score(y_val, y_val_pred)
```

```
#lassification report
```

```
cr = classification_report(y_train, y_train_pred)
```

```
print(cr)
```

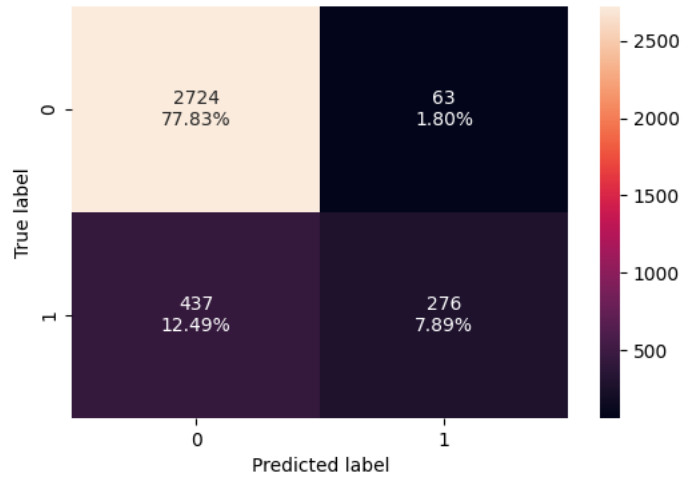
	precision	recall	f1-score	support
0.0	0.80	1.00	0.89	2787
1.0	0.64	0.02	0.04	713

accuracy			0.80	3500
macro avg	0.72	0.51	0.46	3500
weighted avg	0.77	0.80	0.71	3500

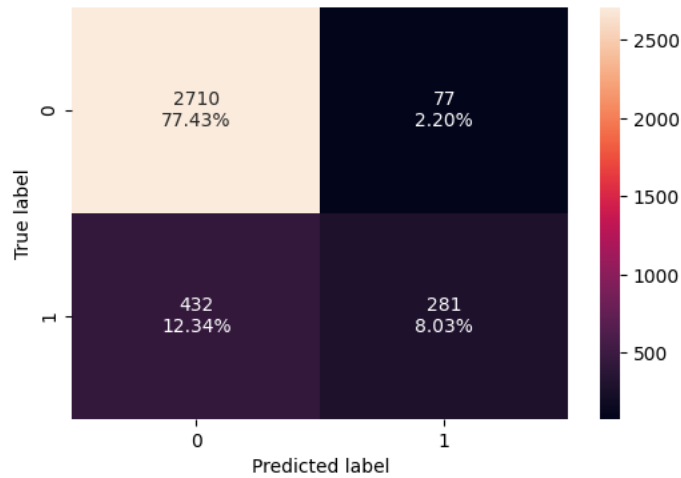
```
#classification report
cr=classification_report(y_val, y_val_pred)  ## Complete the code to
check the model's performance on the validation set
print(cr)
```

	precision	recall	f1-score	support
0.0	0.80	1.00	0.89	2787
1.0	0.73	0.02	0.04	713
accuracy			0.80	3500
macro avg	0.76	0.51	0.47	3500
weighted avg	0.78	0.80	0.72	3500

```
make_confusion_matrix(y_train, y_train_pred)
```



```
make_confusion_matrix(y_val, y_val_pred)  ## Complete the code to
check the model's performance on the validation set
```



A lot of issues, too much class imbalance, isnt really looking at minority class, not fruitful

```
# model_0_train_perf = model_performance_classification(model_0,
#                                                       X_train, y_train)
# model_0_train_perf

# model_0_valid_perf = model_performance_classification(model_0,
#                                                       X_val, y_val)
# model_0_valid_perf
```

Model Performance Improvement

Neural Network with Adam Optimizer

```
backend.clear_session()
#Fixing the seed for random number generators so that we can ensure we
#receive the same output everytime

np.random.seed(2)
random.seed(2)
tf.random.set_seed(2)

#Initializing the neural network
model_1 = Sequential()
model_1.add(Dense(16,activation="relu",input_dim=X_train.shape[1]))
model_1.add(Dropout(0.3))
model_1.add(Dense(9,activation="relu"))
model_1.add(Dropout(0.2))
model_1.add(Dense(1,activation="sigmoid"))

optimizer = tf.keras.optimizers.Adam() # defining Adam as the
#optimizer to be used
metric = keras.metrics.Recall()
model_1.compile(loss='binary_crossentropy', optimizer=optimizer,
               metrics=[metric])
model_1.summary()
```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 16)	
dropout (Dropout)	(None, 16)	
dense_1 (Dense)	(None, 9)	
dropout_1 (Dropout)	(None, 9)	
dense_2 (Dense)	(None, 1)	



Total params: 355 (1.39 KB)

Trainable params: 355 (1.39 KB)

Non-trainable params: 0 (0.00 B)

```
#Fitting the ANN
history_1 = model_1.fit(
    X_train,y_train,
    batch_size=25, ## Complete the code to specify the batch size to
    #use
    validation_data=(X_val,y_val),
    epochs=55, ## Complete the code to specify the number of epochs
    verbose=1
)
```

Epoch 1/55
140/140 ————— 3s 7ms/step - loss: 0.7791 - recall: 0.4714
- val_loss: 0.5284 - val_recall: 0.0449

Epoch 2/55
140/140 ————— 1s 5ms/step - loss: 0.5582 - recall: 0.1052
- val_loss: 0.4878 - val_recall: 0.0323

Epoch 3/55
140/140 ————— 1s 5ms/step - loss: 0.5219 - recall: 0.0939
- val_loss: 0.4658 - val_recall: 0.0477

Epoch 4/55
140/140 ————— 1s 5ms/step - loss: 0.4928 - recall: 0.0870
- val_loss: 0.4531 - val_recall: 0.0589

Epoch 5/55
140/140 ————— 1s 4ms/step - loss: 0.4772 - recall: 0.0831
- val_loss: 0.4451 - val_recall: 0.0799

Epoch 6/55
140/140 ————— 1s 5ms/step - loss: 0.4639 - recall: 0.0994
- val_loss: 0.4394 - val_recall: 0.0884

Epoch 7/55
140/140 ————— 1s 5ms/step - loss: 0.4635 - recall: 0.0846
- val_loss: 0.4334 - val_recall: 0.0982

Epoch 8/55
140/140 ————— 1s 5ms/step - loss: 0.4584 - recall: 0.1388
- val_loss: 0.4309 - val_recall: 0.1024

Epoch 9/55
140/140 ————— 2s 7ms/step - loss: 0.4565 - recall: 0.1252
- val_loss: 0.4276 - val_recall: 0.1220

Epoch 10/55
140/140 ————— 1s 9ms/step - loss: 0.4423 - recall: 0.1445
- val_loss: 0.4247 - val_recall: 0.1318

Epoch 11/55
140/140 ————— 1s 8ms/step - loss: 0.4480 - recall: 0.1238
- val_loss: 0.4200 - val_recall: 0.1641

Epoch 12/55
140/140 ————— 1s 7ms/step - loss: 0.4359 - recall: 0.1343
- val_loss: 0.4158 - val_recall: 0.2020

Epoch 13/55
140/140 ————— 1s 5ms/step - loss: 0.4357 - recall: 0.1907
- val_loss: 0.4136 - val_recall: 0.1879

Epoch 14/55
140/140 ————— 1s 5ms/step - loss: 0.4283 - recall: 0.1941
- val_loss: 0.4105 - val_recall: 0.2132

Epoch 15/55
140/140 ————— 1s 5ms/step - loss: 0.4326 - recall: 0.2111
- val_loss: 0.4085 - val_recall: 0.2118

Epoch 16/55
140/140 ————— 1s 5ms/step - loss: 0.4286 - recall: 0.2286
- val_loss: 0.4072 - val_recall: 0.2244

Epoch 17/55
140/140 ————— 1s 5ms/step - loss: 0.4241 - recall: 0.2080
- val_loss: 0.4033 - val_recall: 0.2370

Epoch 18/55
140/140 ————— 1s 5ms/step - loss: 0.4298 - recall: 0.2295
- val_loss: 0.4018 - val_recall: 0.2356

Epoch 19/55
140/140 ————— 1s 5ms/step - loss: 0.4357 - recall: 0.2163
- val_loss: 0.4001 - val_recall: 0.2384

Epoch 20/55
140/140 ————— 1s 5ms/step - loss: 0.4203 - recall: 0.2158
- val_loss: 0.3980 - val_recall: 0.2454

Epoch 21/55

140/140 ————— 1s 4ms/step - loss: 0.4104 - recall: 0.2403
- val_loss: 0.3944 - val_recall: 0.2679
Epoch 22/55
140/140 ————— 1s 8ms/step - loss: 0.4216 - recall: 0.2181
- val_loss: 0.3919 - val_recall: 0.2707
Epoch 23/55
140/140 ————— 1s 9ms/step - loss: 0.4152 - recall: 0.2289
- val_loss: 0.3893 - val_recall: 0.2721
Epoch 24/55
140/140 ————— 1s 9ms/step - loss: 0.4086 - recall: 0.2687
- val_loss: 0.3878 - val_recall: 0.2637
Epoch 25/55
140/140 ————— 2s 5ms/step - loss: 0.4146 - recall: 0.2452
- val_loss: 0.3847 - val_recall: 0.2875
Epoch 26/55
140/140 ————— 1s 5ms/step - loss: 0.4083 - recall: 0.2821
- val_loss: 0.3826 - val_recall: 0.2791
Epoch 27/55
140/140 ————— 1s 4ms/step - loss: 0.4043 - recall: 0.2842
- val_loss: 0.3792 - val_recall: 0.3114
Epoch 28/55
140/140 ————— 1s 4ms/step - loss: 0.4111 - recall: 0.2581
- val_loss: 0.3782 - val_recall: 0.3128
Epoch 29/55
140/140 ————— 1s 5ms/step - loss: 0.3942 - recall: 0.3006
- val_loss: 0.3753 - val_recall: 0.3268
Epoch 30/55
140/140 ————— 1s 9ms/step - loss: 0.3933 - recall: 0.3209
- val_loss: 0.3743 - val_recall: 0.3324
Epoch 31/55
140/140 ————— 3s 11ms/step - loss: 0.4007 - recall:
0.2773 - val_loss: 0.3733 - val_recall: 0.3394
Epoch 32/55
140/140 ————— 2s 12ms/step - loss: 0.3962 - recall:
0.3032 - val_loss: 0.3736 - val_recall: 0.3520
Epoch 33/55
140/140 ————— 4s 21ms/step - loss: 0.3931 - recall:
0.3096 - val_loss: 0.3738 - val_recall: 0.3436
Epoch 34/55
140/140 ————— 5s 20ms/step - loss: 0.3851 - recall:
0.3263 - val_loss: 0.3718 - val_recall: 0.3562
Epoch 35/55
140/140 ————— 3s 17ms/step - loss: 0.4016 - recall:
0.3057 - val_loss: 0.3710 - val_recall: 0.3562
Epoch 36/55
140/140 ————— 2s 13ms/step - loss: 0.3996 - recall:
0.3071 - val_loss: 0.3697 - val_recall: 0.3576
Epoch 37/55
140/140 ————— 3s 15ms/step - loss: 0.3817 - recall:
0.3463 - val_loss: 0.3682 - val_recall: 0.3647
Epoch 38/55
140/140 ————— 1s 9ms/step - loss: 0.3890 - recall: 0.3238
- val_loss: 0.3672 - val_recall: 0.3815
Epoch 39/55
140/140 ————— 2s 5ms/step - loss: 0.3835 - recall: 0.3635
- val_loss: 0.3677 - val_recall: 0.3478
Epoch 40/55
140/140 ————— 1s 5ms/step - loss: 0.3915 - recall: 0.3143
- val_loss: 0.3666 - val_recall: 0.3604
Epoch 41/55
140/140 ————— 1s 4ms/step - loss: 0.3940 - recall: 0.3216

```

- val_loss: 0.3670 - val_recall: 0.3633
Epoch 42/55
140/140 ————— 1s 5ms/step - loss: 0.3949 - recall: 0.3177
- val_loss: 0.3665 - val_recall: 0.3647
Epoch 43/55
140/140 ————— 1s 5ms/step - loss: 0.3848 - recall: 0.3289
- val_loss: 0.3650 - val_recall: 0.3871
Epoch 44/55
140/140 ————— 1s 5ms/step - loss: 0.3839 - recall: 0.3409
- val_loss: 0.3657 - val_recall: 0.3773
Epoch 45/55
140/140 ————— 1s 5ms/step - loss: 0.3946 - recall: 0.3366
- val_loss: 0.3663 - val_recall: 0.3576
Epoch 46/55
140/140 ————— 2s 8ms/step - loss: 0.3914 - recall: 0.3380
- val_loss: 0.3658 - val_recall: 0.3801
Epoch 47/55
140/140 ————— 1s 8ms/step - loss: 0.3811 - recall: 0.3529
- val_loss: 0.3632 - val_recall: 0.4025
Epoch 48/55
140/140 ————— 1s 5ms/step - loss: 0.3903 - recall: 0.3532
- val_loss: 0.3631 - val_recall: 0.3829
Epoch 49/55
140/140 ————— 1s 5ms/step - loss: 0.3832 - recall: 0.3719
- val_loss: 0.3619 - val_recall: 0.3871
Epoch 50/55
140/140 ————— 1s 4ms/step - loss: 0.3924 - recall: 0.3438
- val_loss: 0.3615 - val_recall: 0.3983
Epoch 51/55
140/140 ————— 1s 5ms/step - loss: 0.3837 - recall: 0.3512
- val_loss: 0.3625 - val_recall: 0.3604
Epoch 52/55
140/140 ————— 1s 5ms/step - loss: 0.3836 - recall: 0.3619
- val_loss: 0.3613 - val_recall: 0.3829
Epoch 53/55
140/140 ————— 1s 4ms/step - loss: 0.3814 - recall: 0.3660
- val_loss: 0.3608 - val_recall: 0.3955
Epoch 54/55
140/140 ————— 1s 4ms/step - loss: 0.3766 - recall: 0.3729
- val_loss: 0.3612 - val_recall: 0.3815
Epoch 55/55
140/140 ————— 1s 5ms/step - loss: 0.3800 - recall: 0.3494
- val_loss: 0.3604 - val_recall: 0.3941

```

```
#Plotting Train Loss vs Validation Loss
```

```

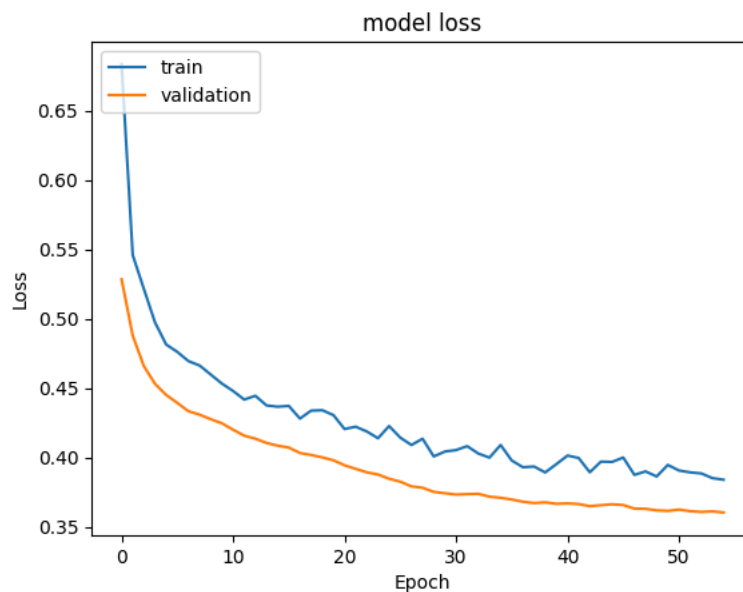
plt.plot(history_1.history['loss'])
plt.plot(history_1.history['val_loss'])
plt.title('model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()

```

```

#this means there is some overfitting happening, model is not quite
there yet, more layers and neurons should be added to close
the gap

```

```
#Plotting Train recall vs Validation recall
plt.plot(history_1.history['recall'])
plt.plot(history_1.history['val_recall'])
plt.title('model recall')
plt.ylabel('Recall')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```

#some irregularities in this, validation set is overtaking the training set, its recall is improving slightly, smaller dataset for validation could be leading to this



```
#Predicting the results using 0.5 as the threshold
y_train_pred = model_1.predict(x_train)
y_train_pred = (y_train_pred > 0.5)
y_train_pred
```

110/110 ————— 0s 2ms/step

```
array([[False],
       [False],
       [False],
```

```

...,
[False],
[False],
[False]])

#Predicting the results using 0.5 as the threshold
y_val_pred = model_1.predict(X_val)
y_val_pred = (y_val_pred > 0.5)
y_val_pred

110/110 ————— 0s 2ms/step

array([[False],
       [False],
       [False],
       ...,
       [False],
       [False],
       [False]])

model_name = "NN with Adam"

train_metric_df.loc[model_name] = recall_score(y_train,y_train_pred)
valid_metric_df.loc[model_name] = recall_score(y_val,y_val_pred)

#classification report
cr=classification_report(y_train,y_train_pred)
print(cr)

           precision    recall  f1-score   support

    0.0         0.86      0.98      0.92        2787
    1.0         0.81      0.39      0.52         713

 accuracy                   0.86        3500
 macro avg              0.84      0.68      0.72        3500
weighted avg              0.85      0.86      0.84        3500

#classification report
cr=classification_report(y_val,y_val_pred)
print(cr)

#too much class imbalance number wise but looking at graph its getting
better

           precision    recall  f1-score   support

    0.0         0.86      0.97      0.91        2787
    1.0         0.78      0.39      0.52         713

 accuracy                   0.85        3500
 macro avg              0.82      0.68      0.72        3500
weighted avg              0.85      0.85      0.83        3500

```

Neural Network with Adam Optimizer and Dropout

```

backend.clear_session()
#Fixing the seed for random number generators so that we can ensure we
receive the same output everytime
np.random.seed(2)
random.seed(2)
tf.random.set_seed(2)

```

```

#Initializing the neural network
model_2 = Sequential()

#Adding the input layer with 32 neurons and relu as activation
function
model_2.add(Dense(32, activation='relu', input_dim=X_train.shape[1]))

# Adding dropout with ratio of 0.3
model_2.add(Dropout(0.3))

# Adding a hidden layer with 64 neurons and relu activation
model_2.add(Dense(64, activation='relu'))

# Adding another hidden layer with 32 neurons and relu activation
model_2.add(Dense(32, activation='relu'))

# Adding dropout with ratio of 0.1
model_2.add(Dropout(0.2))

# Adding a hidden layer with 16 neurons and relu activation
model_2.add(Dense(16, activation='relu'))

# Adding the output layer
# Assuming this is a binary classification problem
model_2.add(Dense(1, activation='sigmoid'))

optimizer = tf.keras.optimizers.Adam()    # defining Adam as the
optimizer to be used
metric = keras.metrics.Recall()
model_2.compile(loss='binary_crossentropy', optimizer=optimizer,
metrics=[metric])
model_2.summary()

```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 32)	
dropout (Dropout)	(None, 32)	
dense_1 (Dense)	(None, 64)	
dense_2 (Dense)	(None, 32)	
dropout_1 (Dropout)	(None, 32)	
dense_3 (Dense)	(None, 16)	
dense_4 (Dense)	(None, 1)	



Total params: 5,121 (20.00 KB)

Trainable params: 5,121 (20.00 KB)

Non-trainable params: 0 (0.00 B)

```

history_2 = model_2.fit(
    X_train,y_train,
    batch_size=25, ##Complete the code to specify the batch size.
    epochs=55, ##Complete the code to specify the # of epochs.
    verbose=1,
    validation_data=(X_val,y_val)
)

```

Epoch 1/55

140/140 ————— 4s 12ms/step - loss: 0.5403 - recall:

0.0492 - val_loss: 0.4537 - val_recall: 0.0000e+00
Epoch 2/55
140/140 ————— 2s 5ms/step - loss: 0.4666 - recall: 0.0078
- val_loss: 0.4385 - val_recall: 0.0014
Epoch 3/55
140/140 ————— 1s 5ms/step - loss: 0.4499 - recall: 0.0625
- val_loss: 0.4303 - val_recall: 0.0729
Epoch 4/55
140/140 ————— 1s 5ms/step - loss: 0.4361 - recall: 0.1257
- val_loss: 0.4161 - val_recall: 0.2749
Epoch 5/55
140/140 ————— 1s 5ms/step - loss: 0.4297 - recall: 0.2051
- val_loss: 0.4020 - val_recall: 0.2959
Epoch 6/55
140/140 ————— 1s 5ms/step - loss: 0.4099 - recall: 0.2430
- val_loss: 0.3840 - val_recall: 0.3576
Epoch 7/55
140/140 ————— 1s 5ms/step - loss: 0.4049 - recall: 0.2871
- val_loss: 0.3755 - val_recall: 0.3380
Epoch 8/55
140/140 ————— 1s 5ms/step - loss: 0.3964 - recall: 0.2871
- val_loss: 0.3697 - val_recall: 0.4123
Epoch 9/55
140/140 ————— 1s 5ms/step - loss: 0.3783 - recall: 0.3359
- val_loss: 0.3629 - val_recall: 0.3899
Epoch 10/55
140/140 ————— 1s 5ms/step - loss: 0.3910 - recall: 0.3131
- val_loss: 0.3601 - val_recall: 0.4025
Epoch 11/55
140/140 ————— 2s 9ms/step - loss: 0.3787 - recall: 0.3310
- val_loss: 0.3614 - val_recall: 0.4208
Epoch 12/55
140/140 ————— 1s 9ms/step - loss: 0.3726 - recall: 0.3769
- val_loss: 0.3578 - val_recall: 0.4123
Epoch 13/55
140/140 ————— 2s 5ms/step - loss: 0.3869 - recall: 0.3553
- val_loss: 0.3579 - val_recall: 0.4039
Epoch 14/55
140/140 ————— 1s 4ms/step - loss: 0.3763 - recall: 0.3755
- val_loss: 0.3536 - val_recall: 0.3997
Epoch 15/55
140/140 ————— 1s 5ms/step - loss: 0.3639 - recall: 0.3804
- val_loss: 0.3524 - val_recall: 0.4278
Epoch 16/55
140/140 ————— 1s 5ms/step - loss: 0.3769 - recall: 0.3785
- val_loss: 0.3527 - val_recall: 0.3913
Epoch 17/55
140/140 ————— 1s 5ms/step - loss: 0.3746 - recall: 0.3553
- val_loss: 0.3572 - val_recall: 0.4109
Epoch 18/55
140/140 ————— 1s 5ms/step - loss: 0.3698 - recall: 0.3775
- val_loss: 0.3539 - val_recall: 0.4432
Epoch 19/55
140/140 ————— 1s 5ms/step - loss: 0.3726 - recall: 0.4028
- val_loss: 0.3550 - val_recall: 0.4208
Epoch 20/55
140/140 ————— 1s 5ms/step - loss: 0.3596 - recall: 0.4101
- val_loss: 0.3505 - val_recall: 0.4614
Epoch 21/55
140/140 ————— 1s 5ms/step - loss: 0.3506 - recall: 0.4230
- val_loss: 0.3516 - val_recall: 0.4165

Epoch 22/55
140/140 ————— 1s 9ms/step - loss: 0.3607 - recall: 0.3614
- val_loss: 0.3496 - val_recall: 0.4306

Epoch 23/55
140/140 ————— 1s 9ms/step - loss: 0.3563 - recall: 0.4166
- val_loss: 0.3481 - val_recall: 0.4222

Epoch 24/55
140/140 ————— 1s 8ms/step - loss: 0.3565 - recall: 0.4183
- val_loss: 0.3464 - val_recall: 0.4432

Epoch 25/55
140/140 ————— 1s 5ms/step - loss: 0.3596 - recall: 0.4187
- val_loss: 0.3476 - val_recall: 0.4404

Epoch 26/55
140/140 ————— 1s 5ms/step - loss: 0.3553 - recall: 0.4156
- val_loss: 0.3466 - val_recall: 0.4460

Epoch 27/55
140/140 ————— 1s 5ms/step - loss: 0.3577 - recall: 0.4473
- val_loss: 0.3462 - val_recall: 0.4600

Epoch 28/55
140/140 ————— 1s 5ms/step - loss: 0.3471 - recall: 0.4348
- val_loss: 0.3488 - val_recall: 0.4348

Epoch 29/55
140/140 ————— 1s 5ms/step - loss: 0.3511 - recall: 0.4145
- val_loss: 0.3472 - val_recall: 0.4474

Epoch 30/55
140/140 ————— 1s 5ms/step - loss: 0.3477 - recall: 0.4408
- val_loss: 0.3484 - val_recall: 0.4334

Epoch 31/55
140/140 ————— 1s 5ms/step - loss: 0.3563 - recall: 0.4264
- val_loss: 0.3487 - val_recall: 0.4432

Epoch 32/55
140/140 ————— 1s 6ms/step - loss: 0.3521 - recall: 0.4516
- val_loss: 0.3487 - val_recall: 0.4362

Epoch 33/55
140/140 ————— 1s 5ms/step - loss: 0.3475 - recall: 0.4221
- val_loss: 0.3474 - val_recall: 0.4390

Epoch 34/55
140/140 ————— 2s 9ms/step - loss: 0.3499 - recall: 0.4322
- val_loss: 0.3504 - val_recall: 0.4432

Epoch 35/55
140/140 ————— 1s 9ms/step - loss: 0.3416 - recall: 0.4460
- val_loss: 0.3483 - val_recall: 0.4867

Epoch 36/55
140/140 ————— 1s 10ms/step - loss: 0.3546 - recall:
0.4602 - val_loss: 0.3526 - val_recall: 0.4783

Epoch 37/55
140/140 ————— 2s 5ms/step - loss: 0.3459 - recall: 0.4626
- val_loss: 0.3501 - val_recall: 0.4474

Epoch 38/55
140/140 ————— 1s 5ms/step - loss: 0.3502 - recall: 0.4275
- val_loss: 0.3488 - val_recall: 0.4558

Epoch 39/55
140/140 ————— 1s 5ms/step - loss: 0.3422 - recall: 0.4702
- val_loss: 0.3464 - val_recall: 0.4712

Epoch 40/55
140/140 ————— 1s 5ms/step - loss: 0.3336 - recall: 0.4897
- val_loss: 0.3500 - val_recall: 0.4460

Epoch 41/55
140/140 ————— 1s 5ms/step - loss: 0.3452 - recall: 0.4235
- val_loss: 0.3510 - val_recall: 0.4516

Epoch 42/55

```

140/140 ————— 1s 5ms/step - loss: 0.3465 - recall: 0.4554
- val_loss: 0.3499 - val_recall: 0.4488
Epoch 43/55
140/140 ————— 1s 5ms/step - loss: 0.3373 - recall: 0.4583
- val_loss: 0.3512 - val_recall: 0.4502
Epoch 44/55
140/140 ————— 1s 5ms/step - loss: 0.3420 - recall: 0.4563
- val_loss: 0.3528 - val_recall: 0.4937
Epoch 45/55
140/140 ————— 1s 5ms/step - loss: 0.3412 - recall: 0.4972
- val_loss: 0.3500 - val_recall: 0.4769
Epoch 46/55
140/140 ————— 2s 9ms/step - loss: 0.3338 - recall: 0.4628
- val_loss: 0.3529 - val_recall: 0.4769
Epoch 47/55
140/140 ————— 1s 9ms/step - loss: 0.3497 - recall: 0.4672
- val_loss: 0.3513 - val_recall: 0.4797
Epoch 48/55
140/140 ————— 2s 5ms/step - loss: 0.3292 - recall: 0.4744
- val_loss: 0.3491 - val_recall: 0.4558
Epoch 49/55
140/140 ————— 1s 5ms/step - loss: 0.3478 - recall: 0.4681
- val_loss: 0.3504 - val_recall: 0.4895
Epoch 50/55
140/140 ————— 1s 5ms/step - loss: 0.3395 - recall: 0.4724
- val_loss: 0.3541 - val_recall: 0.4544
Epoch 51/55
140/140 ————— 1s 5ms/step - loss: 0.3412 - recall: 0.4581
- val_loss: 0.3510 - val_recall: 0.4544
Epoch 52/55
140/140 ————— 1s 5ms/step - loss: 0.3326 - recall: 0.4791
- val_loss: 0.3504 - val_recall: 0.4432
Epoch 53/55
140/140 ————— 1s 5ms/step - loss: 0.3360 - recall: 0.4426
- val_loss: 0.3516 - val_recall: 0.4488
Epoch 54/55
140/140 ————— 1s 5ms/step - loss: 0.3363 - recall: 0.4851
- val_loss: 0.3511 - val_recall: 0.4628
Epoch 55/55
140/140 ————— 1s 5ms/step - loss: 0.3462 - recall: 0.4735
- val_loss: 0.3517 - val_recall: 0.4656

```

```

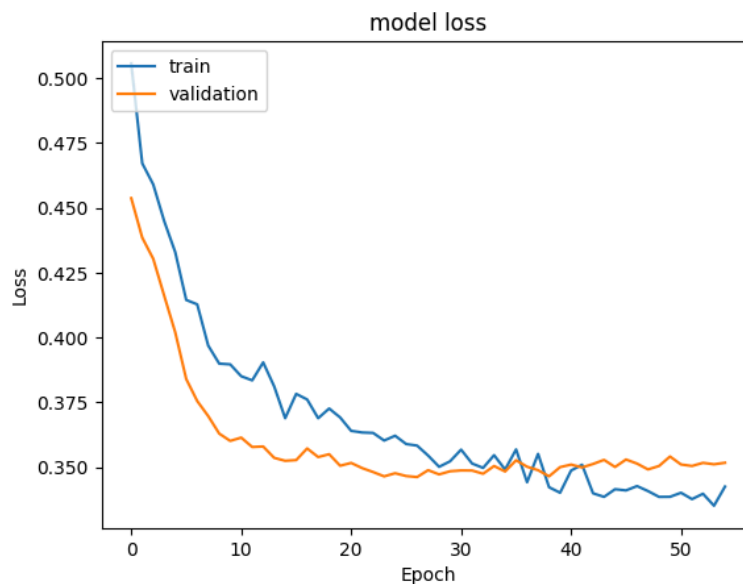
#Plotting Train Loss vs Validation Loss
plt.plot(history_2.history['loss'])
plt.plot(history_2.history['val_loss'])
plt.title('model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()

```

```

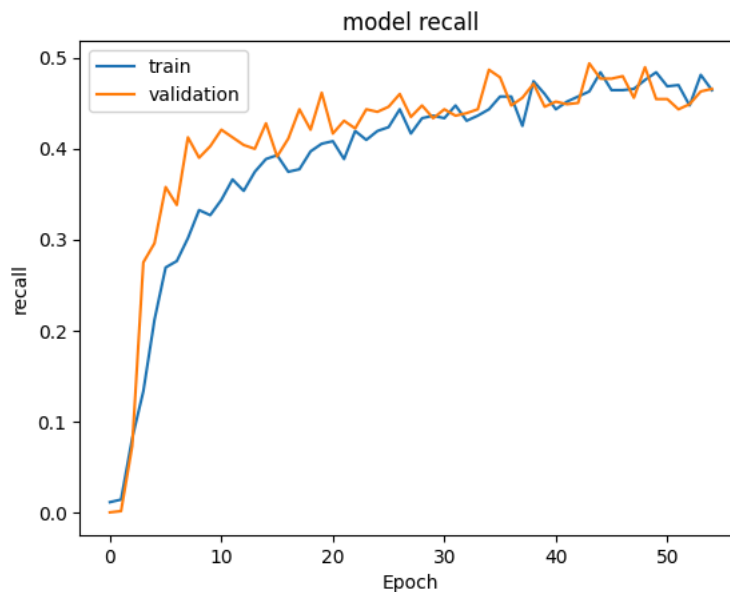
#some overfitting at first but then around epoch 30 its loss levels
out

```



```
#Plotting Train recall vs Validation recall
plt.plot(history_2.history['recall'])
plt.plot(history_2.history['val_recall'])
plt.title('model recall')
plt.ylabel('recall')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```

#really promising graph, some data issues at first for about 8-10 epochs but then its fairly similar, recall improving substainally



```
#Predicting the results using best as a threshold
y_train_pred = model_2.predict(X_train)
y_train_pred = (y_train_pred > 0.5)
y_train_pred
```

110/110 ————— 0s 2ms/step

```
array([[False],
       [False],
       [False],
```

```

...,
[False],
[False],
[False]])

#Predicting the results using 0.5 as the threshold.
y_val_pred = model_2.predict(X_val)
y_val_pred = (y_val_pred > 0.5)
y_val_pred

110/110 ————— 0s 2ms/step

array([[False],
       [ True],
       [False],
       ...,
       [False],
       [False],
       [False]])

model_name = "NN with Adam & Dropout"

train_metric_df.loc[model_name] = recall_score(y_train,y_train_pred)
valid_metric_df.loc[model_name] = recall_score(y_val,y_val_pred)

#classification report
cr=classification_report(y_train,y_train_pred)
print(cr)

              precision    recall  f1-score   support

    0.0         0.89        0.97        0.93        2787
    1.0         0.80        0.53        0.64         713

 accuracy                   0.88        3500
 macro avg              0.84        0.75        0.78        3500
weighted avg              0.87        0.88        0.87        3500

#classification report
cr = classification_report(y_val,y_val_pred) ## Complete the code to
      check the model's performance on the validation set
print(cr)

#looking at both reports, training and validation set recalls are
      bigger even though small ds size

              precision    recall  f1-score   support

    0.0         0.87        0.95        0.91        2787
    1.0         0.71        0.47        0.56         713

 accuracy                   0.85        3500
 macro avg              0.79        0.71        0.74        3500
weighted avg              0.84        0.85        0.84        3500

```

Neural Network with Balanced Data (by applying SMOTE) and SGD Optimizer

```

sm = SMOTE(random_state=42)
#Complete the code to fit SMOTE on the training data.
X_train_smote, y_train_smote = sm.fit_resample(X_train, y_train)

```



```
print('After UpSampling, the shape of train_X:
      {}'.format(X_train_smote.shape))
print('After UpSampling, the shape of train_y: {}
      \n'.format(y_train_smote.shape))
```

After UpSampling, the shape of train_X: (5574, 11)

After UpSampling, the shape of train_y: (5574,)

```
backend.clear_session()
#Fixing the seed for random number generators so that we can ensure we
  receive the same output everytime
np.random.seed(2)
random.seed(2)
tf.random.set_seed(2)

#Initializing the model
model_3 = Sequential()
#Complete the code to add a input layer (specify the # of neurons and
  activation function)
model_3.add(Dense(32,activation='relu',input_dim =
  X_train_smote.shape[1]))
#Complete the code to add a hidden layer (specify the # of neurons and
  activation function)
model_3.add(Dense(16,activation='relu'))
#Complete the code to add a hidden layer (specify the # of neurons and
  activation function)
model_3.add(Dense(9,activation='relu'))
# Complete the code to add the required number of neurons in the
  output layer with a sigmoid activation function.
model_3.add(Dense(1, activation = 'sigmoid'))

optimizer = tf.keras.optimizers.SGD(0.001)    # defining SGD as the
  optimizer to be used
metric = keras.metrics.Recall()
model_3.compile(loss='binary_crossentropy',optimizer=optimizer,metrics=
  [metric])
model_3.summary()
```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 32)	
dense_1 (Dense)	(None, 16)	
dense_2 (Dense)	(None, 9)	
dense_3 (Dense)	(None, 1)	



Total params: 1,075 (4.20 KB)

Trainable params: 1,075 (4.20 KB)

Non-trainable params: 0 (0.00 B)

```
#Fitting the ANN
history_3 = model_3.fit(
    X_train_smote, y_train_smote,
    batch_size=25, ## Complete the code to specify the batch size to
      use
    epochs=55, ## Complete the code to specify the number of epochs
    verbose=1,
    validation_data = (X_val,y_val)
)
```

Epoch 1/55

223/223 ————— 2s 5ms/step - loss: 0.7142 - recall: 0.7516
 - val_loss: 0.7411 - val_recall: 0.7377

Epoch 2/55
223/223 ————— 1s 4ms/step - loss: 0.7076 - recall: 0.7108
- val_loss: 0.7278 - val_recall: 0.6999

Epoch 3/55
223/223 ————— 1s 4ms/step - loss: 0.7023 - recall: 0.6885
- val_loss: 0.7177 - val_recall: 0.6802

Epoch 4/55
223/223 ————— 1s 4ms/step - loss: 0.6978 - recall: 0.6755
- val_loss: 0.7098 - val_recall: 0.6690

Epoch 5/55
223/223 ————— 1s 4ms/step - loss: 0.6939 - recall: 0.6678
- val_loss: 0.7035 - val_recall: 0.6648

Epoch 6/55
223/223 ————— 2s 7ms/step - loss: 0.6903 - recall: 0.6592
- val_loss: 0.6985 - val_recall: 0.6648

Epoch 7/55
223/223 ————— 3s 7ms/step - loss: 0.6870 - recall: 0.6573
- val_loss: 0.6942 - val_recall: 0.6676

Epoch 8/55
223/223 ————— 1s 4ms/step - loss: 0.6837 - recall: 0.6608
- val_loss: 0.6906 - val_recall: 0.6802

Epoch 9/55
223/223 ————— 1s 4ms/step - loss: 0.6805 - recall: 0.6618
- val_loss: 0.6876 - val_recall: 0.6858

Epoch 10/55
223/223 ————— 2s 6ms/step - loss: 0.6774 - recall: 0.6713
- val_loss: 0.6850 - val_recall: 0.6914

Epoch 11/55
223/223 ————— 1s 4ms/step - loss: 0.6744 - recall: 0.6833
- val_loss: 0.6827 - val_recall: 0.6971

Epoch 12/55
223/223 ————— 1s 4ms/step - loss: 0.6714 - recall: 0.6918
- val_loss: 0.6806 - val_recall: 0.6985

Epoch 13/55
223/223 ————— 1s 4ms/step - loss: 0.6685 - recall: 0.6951
- val_loss: 0.6788 - val_recall: 0.7055

Epoch 14/55
223/223 ————— 1s 4ms/step - loss: 0.6656 - recall: 0.6984
- val_loss: 0.6770 - val_recall: 0.7097

Epoch 15/55
223/223 ————— 1s 4ms/step - loss: 0.6627 - recall: 0.7067
- val_loss: 0.6752 - val_recall: 0.7195

Epoch 16/55
223/223 ————— 1s 4ms/step - loss: 0.6600 - recall: 0.7073
- val_loss: 0.6735 - val_recall: 0.7223

Epoch 17/55
223/223 ————— 1s 7ms/step - loss: 0.6573 - recall: 0.7124
- val_loss: 0.6719 - val_recall: 0.7265

Epoch 18/55
223/223 ————— 1s 6ms/step - loss: 0.6546 - recall: 0.7180
- val_loss: 0.6702 - val_recall: 0.7321

Epoch 19/55
223/223 ————— 2s 4ms/step - loss: 0.6520 - recall: 0.7203
- val_loss: 0.6686 - val_recall: 0.7321

Epoch 20/55
223/223 ————— 1s 4ms/step - loss: 0.6494 - recall: 0.7205
- val_loss: 0.6669 - val_recall: 0.7349

Epoch 21/55
223/223 ————— 1s 4ms/step - loss: 0.6469 - recall: 0.7242
- val_loss: 0.6652 - val_recall: 0.7321

Epoch 22/55

223/223 ————— 1s 4ms/step - loss: 0.6444 - recall: 0.7253
- val_loss: 0.6634 - val_recall: 0.7265
Epoch 23/55
223/223 ————— 1s 4ms/step - loss: 0.6419 - recall: 0.7254
- val_loss: 0.6617 - val_recall: 0.7307
Epoch 24/55
223/223 ————— 1s 4ms/step - loss: 0.6395 - recall: 0.7233
- val_loss: 0.6598 - val_recall: 0.7321
Epoch 25/55
223/223 ————— 1s 4ms/step - loss: 0.6371 - recall: 0.7234
- val_loss: 0.6578 - val_recall: 0.7307
Epoch 26/55
223/223 ————— 1s 4ms/step - loss: 0.6348 - recall: 0.7233
- val_loss: 0.6558 - val_recall: 0.7293
Epoch 27/55
223/223 ————— 1s 4ms/step - loss: 0.6324 - recall: 0.7239
- val_loss: 0.6539 - val_recall: 0.7265
Epoch 28/55
223/223 ————— 2s 6ms/step - loss: 0.6302 - recall: 0.7265
- val_loss: 0.6518 - val_recall: 0.7265
Epoch 29/55
223/223 ————— 3s 8ms/step - loss: 0.6280 - recall: 0.7244
- val_loss: 0.6497 - val_recall: 0.7293
Epoch 30/55
223/223 ————— 1s 4ms/step - loss: 0.6259 - recall: 0.7202
- val_loss: 0.6477 - val_recall: 0.7265
Epoch 31/55
223/223 ————— 1s 4ms/step - loss: 0.6238 - recall: 0.7161
- val_loss: 0.6456 - val_recall: 0.7279
Epoch 32/55
223/223 ————— 1s 4ms/step - loss: 0.6218 - recall: 0.7168
- val_loss: 0.6436 - val_recall: 0.7279
Epoch 33/55
223/223 ————— 1s 4ms/step - loss: 0.6198 - recall: 0.7167
- val_loss: 0.6415 - val_recall: 0.7223
Epoch 34/55
223/223 ————— 1s 4ms/step - loss: 0.6179 - recall: 0.7179
- val_loss: 0.6394 - val_recall: 0.7181
Epoch 35/55
223/223 ————— 1s 5ms/step - loss: 0.6160 - recall: 0.7210
- val_loss: 0.6373 - val_recall: 0.7195
Epoch 36/55
223/223 ————— 1s 4ms/step - loss: 0.6142 - recall: 0.7186
- val_loss: 0.6353 - val_recall: 0.7209
Epoch 37/55
223/223 ————— 1s 4ms/step - loss: 0.6124 - recall: 0.7148
- val_loss: 0.6332 - val_recall: 0.7195
Epoch 38/55
223/223 ————— 1s 4ms/step - loss: 0.6106 - recall: 0.7151
- val_loss: 0.6312 - val_recall: 0.7209
Epoch 39/55
223/223 ————— 2s 7ms/step - loss: 0.6089 - recall: 0.7142
- val_loss: 0.6292 - val_recall: 0.7209
Epoch 40/55
223/223 ————— 2s 7ms/step - loss: 0.6072 - recall: 0.7152
- val_loss: 0.6272 - val_recall: 0.7209
Epoch 41/55
223/223 ————— 2s 4ms/step - loss: 0.6056 - recall: 0.7160
- val_loss: 0.6253 - val_recall: 0.7223
Epoch 42/55
223/223 ————— 1s 4ms/step - loss: 0.6039 - recall: 0.7159

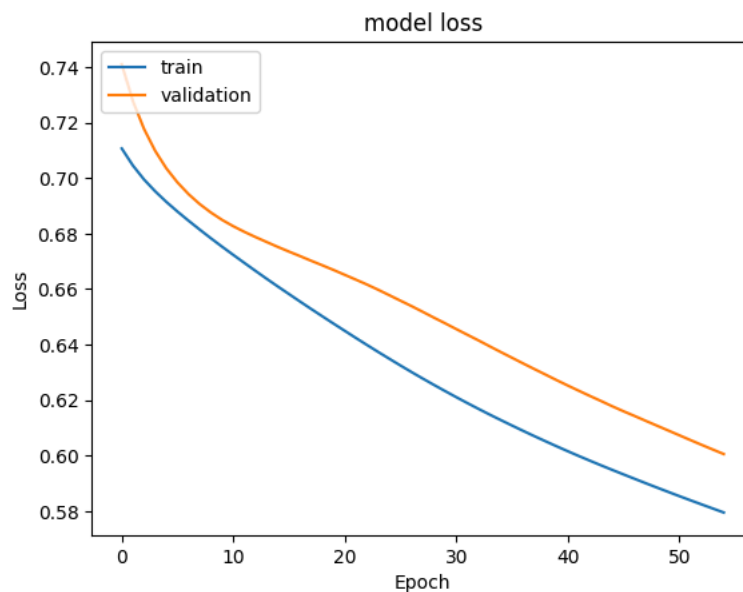
```

- val_loss: 0.6234 - val_recall: 0.7209
Epoch 43/55
223/223 ————— 1s 4ms/step - loss: 0.6024 - recall: 0.7147
- val_loss: 0.6215 - val_recall: 0.7195
Epoch 44/55
223/223 ————— 1s 4ms/step - loss: 0.6008 - recall: 0.7111
- val_loss: 0.6197 - val_recall: 0.7181
Epoch 45/55
223/223 ————— 1s 4ms/step - loss: 0.5993 - recall: 0.7110
- val_loss: 0.6178 - val_recall: 0.7153
Epoch 46/55
223/223 ————— 1s 4ms/step - loss: 0.5978 - recall: 0.7100
- val_loss: 0.6160 - val_recall: 0.7167
Epoch 47/55
223/223 ————— 1s 4ms/step - loss: 0.5963 - recall: 0.7108
- val_loss: 0.6143 - val_recall: 0.7153
Epoch 48/55
223/223 ————— 1s 4ms/step - loss: 0.5948 - recall: 0.7108
- val_loss: 0.6125 - val_recall: 0.7195
Epoch 49/55
223/223 ————— 2s 7ms/step - loss: 0.5934 - recall: 0.7123
- val_loss: 0.6108 - val_recall: 0.7181
Epoch 50/55
223/223 ————— 2s 5ms/step - loss: 0.5919 - recall: 0.7134
- val_loss: 0.6091 - val_recall: 0.7195
Epoch 51/55
223/223 ————— 1s 4ms/step - loss: 0.5905 - recall: 0.7159
- val_loss: 0.6074 - val_recall: 0.7167
Epoch 52/55
223/223 ————— 1s 4ms/step - loss: 0.5891 - recall: 0.7156
- val_loss: 0.6056 - val_recall: 0.7181
Epoch 53/55
223/223 ————— 1s 4ms/step - loss: 0.5877 - recall: 0.7146
- val_loss: 0.6039 - val_recall: 0.7209
Epoch 54/55
223/223 ————— 1s 4ms/step - loss: 0.5863 - recall: 0.7150
- val_loss: 0.6022 - val_recall: 0.7209
Epoch 55/55
223/223 ————— 1s 4ms/step - loss: 0.5849 - recall: 0.7148
- val_loss: 0.6006 - val_recall: 0.7223

#Plotting Train Loss vs Validation Loss
plt.plot(history_3.history['loss'])
plt.plot(history_3.history['val_loss'])
plt.title('model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()

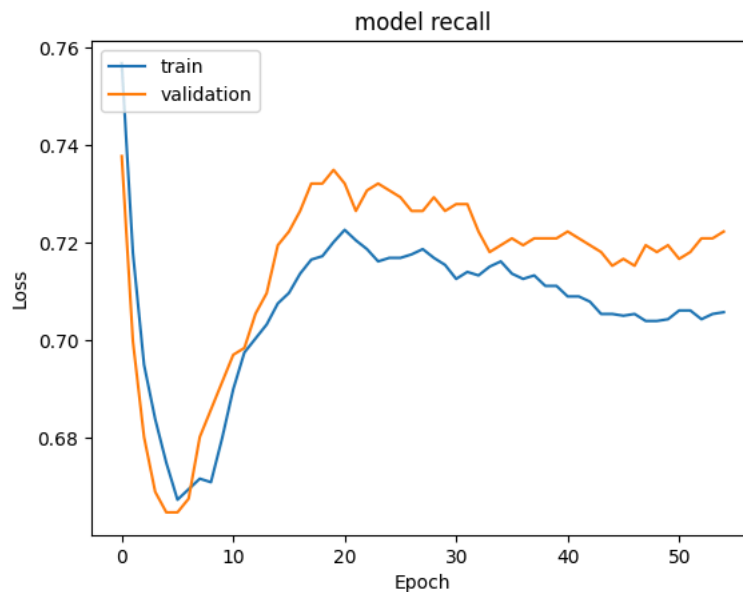
# model is generalizing well but the way the data is being shared
might be the issue here

```



```
#Plotting Train recall vs Validation recall
plt.plot(history_3.history['recall'])
plt.plot(history_3.history['val_recall'])
plt.title('model recall')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```

#graph is promising but towards the end it might be underfitting,
smaller dataset size coupled with FN, FP can lead to higher
false recall which is not good



```
y_train_pred = model_3.predict(x_train_smote)
#Predicting the results using 0.5 as the threshold
y_train_pred = (y_train_pred > 0.5)
y_train_pred
```

175/175 ————— 0s 2ms/step

```
array([[False],
       [False],
       [False],
```

```

...,
[ True],
[ True],
[ True]])

y_val_pred = model_3.predict(X_val)
#Predicting the results using 0.5 as the threshold
y_val_pred = (y_val_pred > 0.5)
y_val_pred

110/110 ————— 0s 2ms/step

array([[False],
       [ True],
       [False],
       ...,
       [False],
       [ True],
       [False]])

model_name = "NN with SMOTE & SGD"

train_metric_df.loc[model_name] =
    recall_score(y_train_smote,y_train_pred)
valid_metric_df.loc[model_name] = recall_score(y_val,y_val_pred)

cr=classification_report(y_train_smote,y_train_pred)
print(cr)

              precision    recall  f1-score   support

    0.0         0.70        0.68        0.69        2787
    1.0         0.69        0.71        0.70        2787

 accuracy                   0.70        5574
 macro avg         0.70        0.70        0.70        5574
weighted avg         0.70        0.70        0.70        5574


cr=classification_report(y_val,y_val_pred) ## Complete the code to
check the model's performance on the validation set
print(cr)

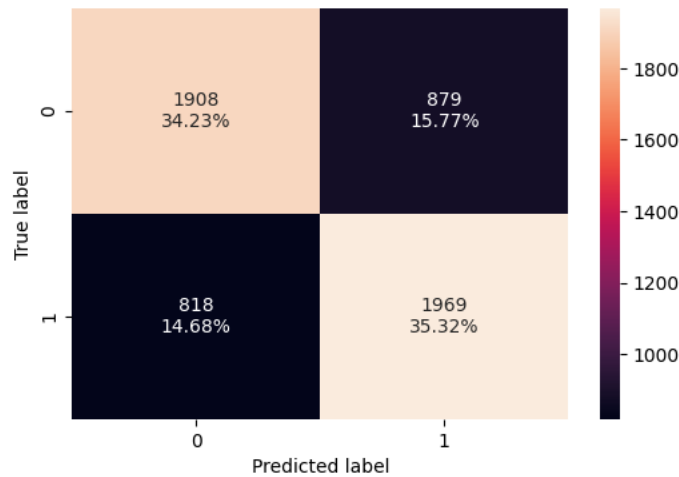
              precision    recall  f1-score   support

    0.0         0.90        0.67        0.77        2787
    1.0         0.36        0.72        0.48         713

 accuracy                   0.68        3500
 macro avg         0.63        0.69        0.62        3500
weighted avg         0.79        0.68        0.71        3500


#Calculating the confusion matrix
make_confusion_matrix(y_train_smote, y_train_pred)

```



```
cr=classification_report(y_val,y_val_pred) ## Complete the code to
check the model's performance on the validation set
print(cr)
```

	precision	recall	f1-score	support
0.0	0.90	0.67	0.77	2787
1.0	0.36	0.72	0.48	713
accuracy			0.68	3500
macro avg	0.63	0.69	0.62	3500
weighted avg	0.79	0.68	0.71	3500

Neural Network with Balanced Data (by applying SMOTE) and Adam Optimizer

```
sm = SMOTE(random_state=42)
#Complete the code to fit SMOTE on the training data.
X_train_smote, y_train_smote = sm.fit_resample(X_train, y_train)
print('After UpSampling, the shape of train_X: {}'
      .format(X_train_smote.shape))
print('After UpSampling, the shape of train_y: {}'
      .format(y_train_smote.shape))

After UpSampling, the shape of train_X: (5574, 11)
After UpSampling, the shape of train_y: (5574,)

backend.clear_session()
#Fixing the seed for random number generators so that we can ensure we
receive the same output everytime
np.random.seed(2)
random.seed(2)
tf.random.set_seed(2)

#Initializing the model
model_4 = Sequential()
#Complete the code to add a input layer (specify the # of neurons and
activation function)
model_4.add(Dense(32,activation='relu',input_dim =
X_train_smote.shape[1]))
model_4.add(Dropout(0.4))
#Complete the code to add a hidden layer (specify the # of neurons and
activation function)
model_4.add(Dense(64,activation='relu'))
#Complete the code to add a hidden layer (specify the # of neurons and
activation function)
model_4.add(Dense(16,activation='relu'))
model_4.add(Dropout(0.4))
```

```
# Complete the code to add the required number of neurons in the
output layer and a suitable activation function.
model_4.add(Dense(1, activation = 'sigmoid'))

optimizer = tf.keras.optimizers.Adam()
metric = keras.metrics.Recall()
model_4.compile(loss='binary_crossentropy',optimizer=optimizer,metrics=
[metric])
model_4.summary()
```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 32)	
dropout (Dropout)	(None, 32)	
dense_1 (Dense)	(None, 64)	
dense_2 (Dense)	(None, 16)	
dropout_1 (Dropout)	(None, 16)	
dense_3 (Dense)	(None, 1)	



Total params: 3,553 (13.88 KB)

Trainable params: 3,553 (13.88 KB)

Non-trainable params: 0 (0.00 B)

#Fitting the ANN

```
# print(history_4.history.keys())
```

```
history_4 = model_4.fit(
    X_train_smote,y_train_smote,
    batch_size=25, ## Complete the code to specify the batch size to
    use
    epochs=55, ## Complete the code to specify the number of epochs
    verbose=1,
    validation_data = (X_val,y_val)
)
```

Epoch 1/55
223/223 ————— 4s 9ms/step - loss: 0.6734 - recall: 0.5729
- val_loss: 0.6190 - val_recall: 0.7658

Epoch 2/55
223/223 ————— 2s 8ms/step - loss: 0.6227 - recall: 0.7012
- val_loss: 0.5786 - val_recall: 0.7784

Epoch 3/55
223/223 ————— 1s 5ms/step - loss: 0.5897 - recall: 0.7356
- val_loss: 0.5650 - val_recall: 0.8050

Epoch 4/55
223/223 ————— 1s 4ms/step - loss: 0.5769 - recall: 0.7391
- val_loss: 0.5295 - val_recall: 0.7812

Epoch 5/55
223/223 ————— 1s 4ms/step - loss: 0.5481 - recall: 0.7663
- val_loss: 0.5140 - val_recall: 0.7840

Epoch 6/55
223/223 ————— 1s 4ms/step - loss: 0.5486 - recall: 0.7573
- val_loss: 0.5067 - val_recall: 0.7756

Epoch 7/55
223/223 ————— 1s 4ms/step - loss: 0.5290 - recall: 0.7666

- val_loss: 0.5199 - val_recall: 0.7952
Epoch 8/55
223/223 ————— 1s 4ms/step - loss: 0.5152 - recall: 0.7791
- val_loss: 0.4988 - val_recall: 0.7686
Epoch 9/55
223/223 ————— 1s 4ms/step - loss: 0.5098 - recall: 0.7686
- val_loss: 0.4943 - val_recall: 0.7728
Epoch 10/55
223/223 ————— 1s 4ms/step - loss: 0.5022 - recall: 0.7927
- val_loss: 0.4741 - val_recall: 0.7518
Epoch 11/55
223/223 ————— 1s 4ms/step - loss: 0.5085 - recall: 0.7794
- val_loss: 0.4986 - val_recall: 0.7714
Epoch 12/55
223/223 ————— 1s 6ms/step - loss: 0.5022 - recall: 0.7896
- val_loss: 0.4859 - val_recall: 0.7616
Epoch 13/55
223/223 ————— 3s 8ms/step - loss: 0.4920 - recall: 0.7870
- val_loss: 0.4750 - val_recall: 0.7602
Epoch 14/55
223/223 ————— 1s 4ms/step - loss: 0.4951 - recall: 0.7841
- val_loss: 0.4949 - val_recall: 0.7714
Epoch 15/55
223/223 ————— 1s 6ms/step - loss: 0.4805 - recall: 0.8087
- val_loss: 0.4988 - val_recall: 0.7574
Epoch 16/55
223/223 ————— 3s 7ms/step - loss: 0.4845 - recall: 0.7876
- val_loss: 0.4830 - val_recall: 0.7630
Epoch 17/55
223/223 ————— 1s 5ms/step - loss: 0.4830 - recall: 0.7798
- val_loss: 0.4737 - val_recall: 0.7560
Epoch 18/55
223/223 ————— 2s 7ms/step - loss: 0.4820 - recall: 0.7837
- val_loss: 0.4853 - val_recall: 0.7532
Epoch 19/55
223/223 ————— 2s 6ms/step - loss: 0.4851 - recall: 0.7999
- val_loss: 0.4808 - val_recall: 0.7602
Epoch 20/55
223/223 ————— 3s 9ms/step - loss: 0.4768 - recall: 0.7954
- val_loss: 0.4753 - val_recall: 0.7447
Epoch 21/55
223/223 ————— 1s 5ms/step - loss: 0.4712 - recall: 0.7885
- val_loss: 0.4840 - val_recall: 0.7588
Epoch 22/55
223/223 ————— 1s 5ms/step - loss: 0.4796 - recall: 0.8088
- val_loss: 0.5019 - val_recall: 0.7560
Epoch 23/55
223/223 ————— 1s 5ms/step - loss: 0.4741 - recall: 0.8015
- val_loss: 0.4817 - val_recall: 0.7574
Epoch 24/55
223/223 ————— 2s 6ms/step - loss: 0.4796 - recall: 0.8070
- val_loss: 0.4727 - val_recall: 0.7447
Epoch 25/55
223/223 ————— 1s 5ms/step - loss: 0.4516 - recall: 0.8128
- val_loss: 0.4898 - val_recall: 0.7574
Epoch 26/55
223/223 ————— 1s 4ms/step - loss: 0.4794 - recall: 0.7989
- val_loss: 0.4859 - val_recall: 0.7574
Epoch 27/55
223/223 ————— 1s 4ms/step - loss: 0.4686 - recall: 0.8003
- val_loss: 0.4728 - val_recall: 0.7574

Epoch 28/55
223/223 ————— 1s 5ms/step - loss: 0.4787 - recall: 0.8005
- val_loss: 0.4911 - val_recall: 0.7588

Epoch 29/55
223/223 ————— 2s 8ms/step - loss: 0.4559 - recall: 0.8131
- val_loss: 0.4704 - val_recall: 0.7489

Epoch 30/55
223/223 ————— 2s 8ms/step - loss: 0.4619 - recall: 0.8109
- val_loss: 0.4927 - val_recall: 0.7546

Epoch 31/55
223/223 ————— 1s 5ms/step - loss: 0.4625 - recall: 0.8144
- val_loss: 0.4865 - val_recall: 0.7433

Epoch 32/55
223/223 ————— 1s 4ms/step - loss: 0.4616 - recall: 0.8146
- val_loss: 0.4846 - val_recall: 0.7532

Epoch 33/55
223/223 ————— 1s 4ms/step - loss: 0.4617 - recall: 0.8085
- val_loss: 0.4670 - val_recall: 0.7321

Epoch 34/55
223/223 ————— 1s 4ms/step - loss: 0.4548 - recall: 0.8088
- val_loss: 0.4730 - val_recall: 0.7391

Epoch 35/55
223/223 ————— 1s 4ms/step - loss: 0.4504 - recall: 0.8162
- val_loss: 0.4606 - val_recall: 0.7405

Epoch 36/55
223/223 ————— 1s 5ms/step - loss: 0.4546 - recall: 0.8204
- val_loss: 0.4868 - val_recall: 0.7574

Epoch 37/55
223/223 ————— 2s 7ms/step - loss: 0.4623 - recall: 0.8252
- val_loss: 0.4898 - val_recall: 0.7588

Epoch 38/55
223/223 ————— 1s 5ms/step - loss: 0.4519 - recall: 0.8132
- val_loss: 0.4825 - val_recall: 0.7546

Epoch 39/55
223/223 ————— 1s 6ms/step - loss: 0.4548 - recall: 0.8128
- val_loss: 0.4663 - val_recall: 0.7518

Epoch 40/55
223/223 ————— 2s 8ms/step - loss: 0.4414 - recall: 0.8207
- val_loss: 0.4737 - val_recall: 0.7518

Epoch 41/55
223/223 ————— 1s 6ms/step - loss: 0.4450 - recall: 0.8300
- val_loss: 0.4630 - val_recall: 0.7532

Epoch 42/55
223/223 ————— 2s 6ms/step - loss: 0.4351 - recall: 0.8256
- val_loss: 0.4720 - val_recall: 0.7377

Epoch 43/55
223/223 ————— 2s 5ms/step - loss: 0.4475 - recall: 0.8217
- val_loss: 0.4873 - val_recall: 0.7602

Epoch 44/55
223/223 ————— 1s 5ms/step - loss: 0.4498 - recall: 0.8257
- val_loss: 0.4741 - val_recall: 0.7574

Epoch 45/55
223/223 ————— 1s 5ms/step - loss: 0.4439 - recall: 0.8363
- val_loss: 0.4797 - val_recall: 0.7363

Epoch 46/55
223/223 ————— 1s 5ms/step - loss: 0.4411 - recall: 0.8317
- val_loss: 0.4751 - val_recall: 0.7574

Epoch 47/55
223/223 ————— 1s 4ms/step - loss: 0.4327 - recall: 0.8302
- val_loss: 0.4803 - val_recall: 0.7714

Epoch 48/55

```

223/223 ————— 2s 7ms/step - loss: 0.4445 - recall: 0.8320
- val_loss: 0.4661 - val_recall: 0.7588
Epoch 49/55
223/223 ————— 2s 7ms/step - loss: 0.4329 - recall: 0.8333
- val_loss: 0.4828 - val_recall: 0.7672
Epoch 50/55
223/223 ————— 1s 5ms/step - loss: 0.4441 - recall: 0.8432
- val_loss: 0.4806 - val_recall: 0.7686
Epoch 51/55
223/223 ————— 1s 4ms/step - loss: 0.4531 - recall: 0.8302
- val_loss: 0.4879 - val_recall: 0.7728
Epoch 52/55
223/223 ————— 1s 4ms/step - loss: 0.4289 - recall: 0.8483
- val_loss: 0.4722 - val_recall: 0.7518
Epoch 53/55
223/223 ————— 1s 5ms/step - loss: 0.4319 - recall: 0.8390
- val_loss: 0.4746 - val_recall: 0.7644
Epoch 54/55
223/223 ————— 1s 5ms/step - loss: 0.4323 - recall: 0.8510
- val_loss: 0.4864 - val_recall: 0.7742
Epoch 55/55
223/223 ————— 1s 5ms/step - loss: 0.4279 - recall: 0.8506
- val_loss: 0.4845 - val_recall: 0.7798

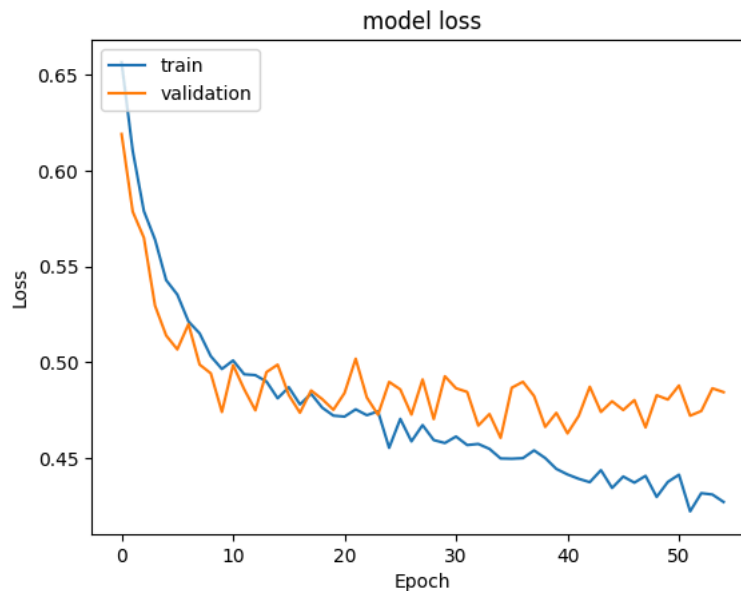
```

```

#Plotting Train Loss vs Validation Loss
plt.plot(history_4.history['loss'])
plt.plot(history_4.history['val_loss'])
plt.title('model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()

```

loss starts strong but finishes weak, could be underfitting data



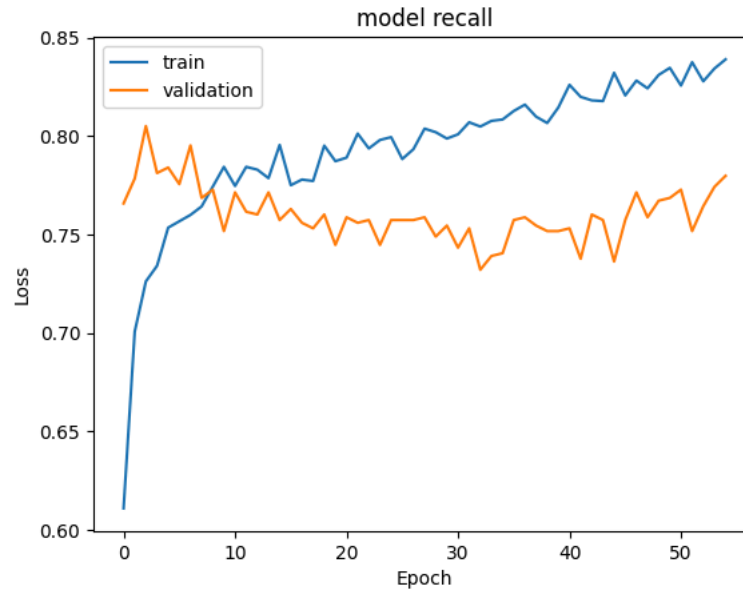
```

#Plotting Train recall vs Validation recall
plt.plot(history_4.history['recall'])
plt.plot(history_4.history['val_recall'])
plt.title('model recall')
plt.ylabel('Recall')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')

```

```
plt.show()
```

```
# recall starts at 0.76, spikes up then levels down a little and ends
    near starting spot, possible overfitting?
```



```
y_train_pred = model_4.predict(X_train_smote)
#Predicting the results using 0.5 as the threshold
y_train_pred = (y_train_pred > 0.5)
y_train_pred
```

```
175/175 ————— 0s 2ms/step
```

```
array([[False],
       [False],
       [False],
       ...,
       [ True],
       [ True],
       [ True]])
```

```
y_train_pred = model_4.predict(X_train_smote)
#Predicting the results using 0.5 as the threshold
y_train_pred = (y_train_pred > 0.5)
y_train_pred
```

```
175/175 ————— 0s 2ms/step
```

```
array([[False],
       [False],
       [False],
       ...,
       [ True],
       [ True],
       [ True]])
```

```
model_name = "NN with SMOTE & Adam"
```

```
train_metric_df.loc[model_name] =
    recall_score(y_train_smote,y_train_pred)
valid_metric_df.loc[model_name] = recall_score(y_val,y_val_pred)

cr=classification_report(y_train_smote,y_train_pred)
print(cr)
```

	precision	recall	f1-score	support
0.0	0.88	0.75	0.81	2787
1.0	0.78	0.89	0.83	2787
accuracy			0.82	5574
macro avg	0.83	0.82	0.82	5574
weighted avg	0.83	0.82	0.82	5574

```
cr=classification_report(y_val,y_val_pred) ## Complete the code to
check the model's performance on the validation set
print(cr)
```

```
#recall for both training, validation and 0 and 1 is looking good, for
0, training starts at .75 and validation is at .67, getting
closer
#for 1 recall starts off strong at .89 and validation is at .72, not
close but 72 is decent
```

	precision	recall	f1-score	support
0.0	0.90	0.67	0.77	2787
1.0	0.36	0.72	0.48	713
accuracy			0.68	3500
macro avg	0.63	0.69	0.62	3500
weighted avg	0.79	0.68	0.71	3500

Neural Network with Balanced Data (by applying SMOTE), Adam Optimizer, and Dropout

```
backend.clear_session()
#Fixing the seed for random number generators so that we can ensure we
receive the same output everytime
np.random.seed(2)
random.seed(2)
tf.random.set_seed(2)

#Initializing the model
model_5 = Sequential()
# Complete the code to add required # of neurons to the input layer
with relu as activation function
model_5.add(Dense(32,activation='relu',input_dim =
X_train_smote.shape[1]))
##https://leetcode.com/discuss/general-
discussion/460599/blind-75-leetcode-questions
#Complete the code to add dropout rate
model_5.add(Dropout(0.4))
# Complete the code to add required # neurons to the hidden layer with
any activation function.
model_5.add(Dense(64,activation='relu'))
# Complete the code to add dropout rate.
model_5.add(Dropout(0.3))
# Adding hidden layer with 8 neurons with relu as activation function
model_5.add(Dense(8,activation='relu'))
# Complete the code to add the required number of neurons in the
output layer with a suitable activation function.
model_5.add(Dense(1, activation = 'sigmoid'))

optimizer = tf.keras.optimizers.Adam()
metric = keras.metrics.Recall()
model_5.compile(loss='binary_crossentropy',optimizer=optimizer,metrics=
[metric])
model_5.summary()

Model: "sequential"
```

Layer (type)	Output Shape	
dense (Dense)	(None, 32)	
dropout (Dropout)	(None, 32)	
dense_1 (Dense)	(None, 64)	
dropout_1 (Dropout)	(None, 64)	
dense_2 (Dense)	(None, 8)	
dense_3 (Dense)	(None, 1)	



Total params: 3,025 (11.82 KB)

Trainable params: 3,025 (11.82 KB)

Non-trainable params: 0 (0.00 B)

```
history_5 = model_5.fit(
    X_train_smote,y_train_smote,
    batch_size=25, ## Complete the code to specify the batch size to
    use
    epochs=55, ## Complete the code to specify the number of epochs
    verbose=1,
    validation_data = (X_val,y_val))
```

Epoch 1/55
 223/223 ————— 5s 8ms/step - loss: 0.6547 - recall: 0.6430
 - val_loss: 0.6164 - val_recall: 0.7700

Epoch 2/55
 223/223 ————— 2s 5ms/step - loss: 0.6222 - recall: 0.6817
 - val_loss: 0.5859 - val_recall: 0.7616

Epoch 3/55
 223/223 ————— 1s 5ms/step - loss: 0.5957 - recall: 0.7191
 - val_loss: 0.5702 - val_recall: 0.7756

Epoch 4/55
 223/223 ————— 1s 5ms/step - loss: 0.5760 - recall: 0.7365
 - val_loss: 0.5490 - val_recall: 0.7784

Epoch 5/55
 223/223 ————— 1s 5ms/step - loss: 0.5537 - recall: 0.7818
 - val_loss: 0.5261 - val_recall: 0.7882

Epoch 6/55
 223/223 ————— 1s 4ms/step - loss: 0.5435 - recall: 0.7782
 - val_loss: 0.5116 - val_recall: 0.7882

Epoch 7/55
 223/223 ————— 1s 4ms/step - loss: 0.5271 - recall: 0.7826
 - val_loss: 0.5084 - val_recall: 0.7784

Epoch 8/55
 223/223 ————— 1s 4ms/step - loss: 0.5099 - recall: 0.7889
 - val_loss: 0.4967 - val_recall: 0.7644

Epoch 9/55
 223/223 ————— 1s 6ms/step - loss: 0.5143 - recall: 0.7764
 - val_loss: 0.4879 - val_recall: 0.7546

Epoch 10/55
 223/223 ————— 3s 7ms/step - loss: 0.4930 - recall: 0.7853
 - val_loss: 0.4868 - val_recall: 0.7475

Epoch 11/55
 223/223 ————— 2s 5ms/step - loss: 0.5089 - recall: 0.7768
 - val_loss: 0.4968 - val_recall: 0.7574

Epoch 12/55
 223/223 ————— 1s 4ms/step - loss: 0.4918 - recall: 0.7911
 - val_loss: 0.4894 - val_recall: 0.7560

Epoch 13/55
223/223 ————— 1s 5ms/step - loss: 0.4978 - recall: 0.7790
- val_loss: 0.4837 - val_recall: 0.7475

Epoch 14/55
223/223 ————— 1s 5ms/step - loss: 0.4881 - recall: 0.7873
- val_loss: 0.4976 - val_recall: 0.7602

Epoch 15/55
223/223 ————— 1s 4ms/step - loss: 0.4780 - recall: 0.7988
- val_loss: 0.5018 - val_recall: 0.7644

Epoch 16/55
223/223 ————— 1s 5ms/step - loss: 0.4854 - recall: 0.7884
- val_loss: 0.4859 - val_recall: 0.7532

Epoch 17/55
223/223 ————— 1s 5ms/step - loss: 0.4901 - recall: 0.7905
- val_loss: 0.4849 - val_recall: 0.7546

Epoch 18/55
223/223 ————— 1s 5ms/step - loss: 0.4847 - recall: 0.7791
- val_loss: 0.4920 - val_recall: 0.7532

Epoch 19/55
223/223 ————— 2s 8ms/step - loss: 0.4713 - recall: 0.8024
- val_loss: 0.4824 - val_recall: 0.7475

Epoch 20/55
223/223 ————— 2s 7ms/step - loss: 0.4824 - recall: 0.7894
- val_loss: 0.4835 - val_recall: 0.7461

Epoch 21/55
223/223 ————— 1s 5ms/step - loss: 0.4686 - recall: 0.7825
- val_loss: 0.4897 - val_recall: 0.7447

Epoch 22/55
223/223 ————— 1s 5ms/step - loss: 0.4679 - recall: 0.7933
- val_loss: 0.4792 - val_recall: 0.7574

Epoch 23/55
223/223 ————— 1s 5ms/step - loss: 0.4666 - recall: 0.7949
- val_loss: 0.4825 - val_recall: 0.7574

Epoch 24/55
223/223 ————— 1s 5ms/step - loss: 0.4632 - recall: 0.7984
- val_loss: 0.4817 - val_recall: 0.7405

Epoch 25/55
223/223 ————— 1s 5ms/step - loss: 0.4553 - recall: 0.8040
- val_loss: 0.4928 - val_recall: 0.7560

Epoch 26/55
223/223 ————— 1s 5ms/step - loss: 0.4679 - recall: 0.8031
- val_loss: 0.4777 - val_recall: 0.7363

Epoch 27/55
223/223 ————— 1s 4ms/step - loss: 0.4668 - recall: 0.7951
- val_loss: 0.4844 - val_recall: 0.7574

Epoch 28/55
223/223 ————— 2s 6ms/step - loss: 0.4664 - recall: 0.7891
- val_loss: 0.4784 - val_recall: 0.7532

Epoch 29/55
223/223 ————— 2s 8ms/step - loss: 0.4574 - recall: 0.8009
- val_loss: 0.4737 - val_recall: 0.7433

Epoch 30/55
223/223 ————— 2s 7ms/step - loss: 0.4652 - recall: 0.7858
- val_loss: 0.4903 - val_recall: 0.7686

Epoch 31/55
223/223 ————— 2s 5ms/step - loss: 0.4633 - recall: 0.7931
- val_loss: 0.4799 - val_recall: 0.7546

Epoch 32/55
223/223 ————— 1s 5ms/step - loss: 0.4511 - recall: 0.8066
- val_loss: 0.4832 - val_recall: 0.7630

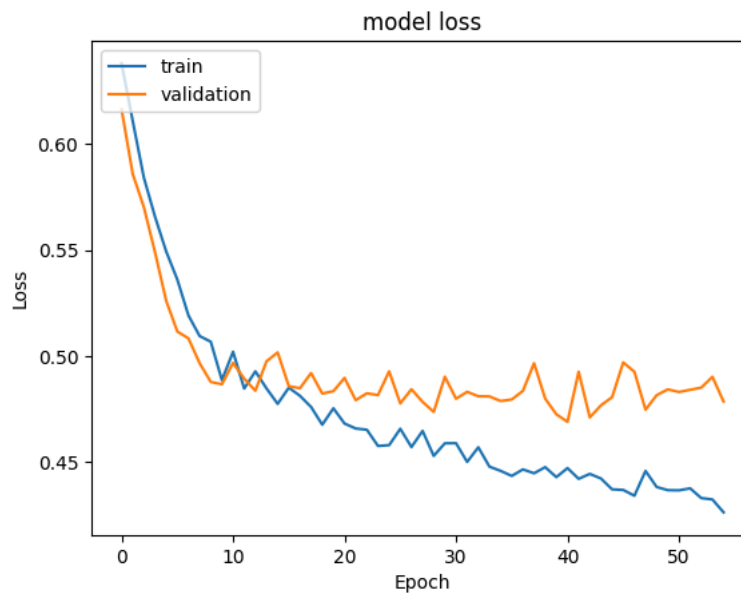
Epoch 33/55

223/223 ————— 1s 5ms/step - loss: 0.4634 - recall: 0.7889
- val_loss: 0.4811 - val_recall: 0.7546
Epoch 34/55
223/223 ————— 1s 5ms/step - loss: 0.4545 - recall: 0.7938
- val_loss: 0.4810 - val_recall: 0.7644
Epoch 35/55
223/223 ————— 1s 5ms/step - loss: 0.4461 - recall: 0.8031
- val_loss: 0.4788 - val_recall: 0.7630
Epoch 36/55
223/223 ————— 1s 5ms/step - loss: 0.4395 - recall: 0.8001
- val_loss: 0.4796 - val_recall: 0.7658
Epoch 37/55
223/223 ————— 1s 5ms/step - loss: 0.4522 - recall: 0.7961
- val_loss: 0.4836 - val_recall: 0.7630
Epoch 38/55
223/223 ————— 1s 6ms/step - loss: 0.4476 - recall: 0.7996
- val_loss: 0.4967 - val_recall: 0.7672
Epoch 39/55
223/223 ————— 2s 7ms/step - loss: 0.4509 - recall: 0.7948
- val_loss: 0.4800 - val_recall: 0.7602
Epoch 40/55
223/223 ————— 2s 4ms/step - loss: 0.4376 - recall: 0.8078
- val_loss: 0.4726 - val_recall: 0.7447
Epoch 41/55
223/223 ————— 1s 5ms/step - loss: 0.4569 - recall: 0.7952
- val_loss: 0.4690 - val_recall: 0.7475
Epoch 42/55
223/223 ————— 1s 5ms/step - loss: 0.4439 - recall: 0.8044
- val_loss: 0.4926 - val_recall: 0.7756
Epoch 43/55
223/223 ————— 1s 5ms/step - loss: 0.4481 - recall: 0.8050
- val_loss: 0.4711 - val_recall: 0.7461
Epoch 44/55
223/223 ————— 1s 5ms/step - loss: 0.4442 - recall: 0.8012
- val_loss: 0.4768 - val_recall: 0.7658
Epoch 45/55
223/223 ————— 1s 4ms/step - loss: 0.4382 - recall: 0.7996
- val_loss: 0.4806 - val_recall: 0.7574
Epoch 46/55
223/223 ————— 1s 4ms/step - loss: 0.4424 - recall: 0.8034
- val_loss: 0.4970 - val_recall: 0.7826
Epoch 47/55
223/223 ————— 1s 4ms/step - loss: 0.4333 - recall: 0.8123
- val_loss: 0.4927 - val_recall: 0.7798
Epoch 48/55
223/223 ————— 1s 6ms/step - loss: 0.4477 - recall: 0.8100
- val_loss: 0.4748 - val_recall: 0.7658
Epoch 49/55
223/223 ————— 3s 8ms/step - loss: 0.4429 - recall: 0.7986
- val_loss: 0.4816 - val_recall: 0.7728
Epoch 50/55
223/223 ————— 1s 4ms/step - loss: 0.4427 - recall: 0.8108
- val_loss: 0.4843 - val_recall: 0.7784
Epoch 51/55
223/223 ————— 1s 4ms/step - loss: 0.4407 - recall: 0.8098
- val_loss: 0.4831 - val_recall: 0.7742
Epoch 52/55
223/223 ————— 1s 4ms/step - loss: 0.4418 - recall: 0.8082
- val_loss: 0.4841 - val_recall: 0.7658
Epoch 53/55
223/223 ————— 1s 5ms/step - loss: 0.4420 - recall: 0.8101


```
- val_loss: 0.4852 - val_recall: 0.7728
Epoch 54/55
223/223 ————— 1s 4ms/step - loss: 0.4380 - recall: 0.8182
- val_loss: 0.4903 - val_recall: 0.7728
Epoch 55/55
223/223 ————— 1s 5ms/step - loss: 0.4274 - recall: 0.8189
- val_loss: 0.4786 - val_recall: 0.7686
```

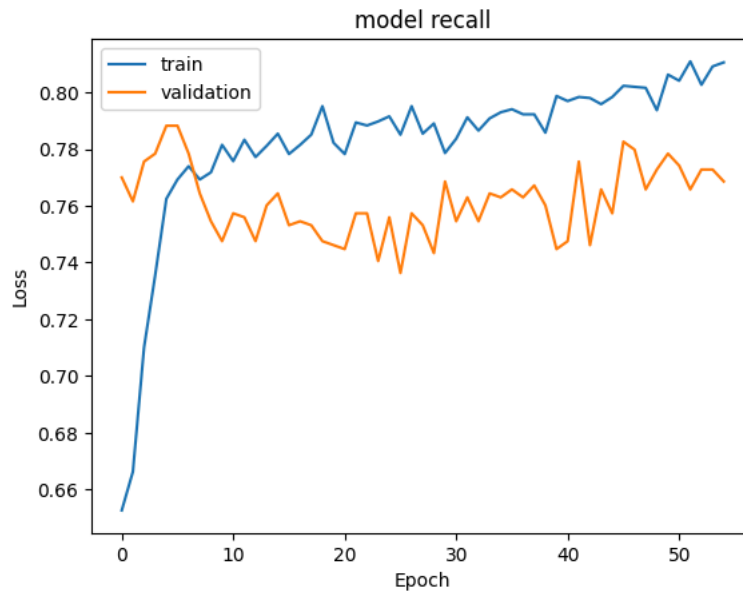
```
#Plotting Train Loss vs Validation Loss
plt.plot(history_5.history['loss'])
plt.plot(history_5.history['val_loss'])
plt.title('model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```

#loss should be lower to 0.3 but is at 0.49



```
#Plotting Train recall vs Validation recall
plt.plot(history_5.history['recall'])
plt.plot(history_5.history['val_recall'])
plt.title('model recall')
plt.ylabel('Recall')
plt.xlabel('Epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```

#recall is high, stoops down but never gets too close to training



```

y_train_pred = model_5.predict(X_train_smote)
#Predicting the results using 0.5 as the threshold
y_train_pred = (y_train_pred > 0.5)
y_train_pred

175/175 ————— 0s 2ms/step

array([[False],
       [False],
       [False],
       ...,
       [ True],
       [False],
       [ True]])

y_val_pred = model_5.predict(X_val)
#Predicting the results using 0.5 as the threshold
y_val_pred = (y_val_pred > 0.5)
y_val_pred

110/110 ————— 0s 1ms/step

array([[False],
       [ True],
       [False],
       ...,
       [False],
       [ True],
       [False]])

model_name = "NN with SMOTE,Adam & Dropout"

train_metric_df.loc[model_name] =
    recall_score(y_train_smote,y_train_pred)
valid_metric_df.loc[model_name] = recall_score(y_val,y_val_pred)

cr=classification_report(y_train_smote,y_train_pred)
print(cr)

```

	precision	recall	f1-score	support
0.0	0.85	0.79	0.82	2787
1.0	0.80	0.86	0.83	2787

accuracy			0.82	5574
macro avg	0.82	0.82	0.82	5574
weighted avg	0.82	0.82	0.82	5574

```
#classification report
cr=classification_report(y_val,y_val_pred) ## Complete the code to
    check the model's performance on the validation set
print(cr)

#model has gotten better at reading members who are staying in class 0
    with a recall of .76
#model has made a jump at recall from the last model for validation
    set meaning the models methods are working in recalling TP n
    all
```

	precision	recall	f1-score	support
0.0	0.93	0.76	0.84	2787
1.0	0.45	0.77	0.57	713
accuracy			0.76	3500
macro avg	0.69	0.77	0.70	3500
weighted avg	0.83	0.76	0.78	3500

Model Performance Comparison and Final Model Selection

```
print("Training performance comparison")
train_metric_df

#highest recall was model_4 with a 0.89 recall, which is very solid

Training performance comparison
```

	recall
NN with SGD	0.019635
NN with Adam	0.387097
NN with Adam & Dropout	0.528752
NN with SMOTE & SGD	0.706494
NN with SMOTE & Adam	0.893793
NN with SMOTE,Adam & Dropout	0.855759

```
print("Validation set performance comparison")
valid_metric_df

#model_5 had the highest recall in validation, which is what we need
    to see since that is how our model is behaving after seeing
    the data, 0.76 is good, not amazing but decent

Validation set performance comparison
```

	recall
NN with SGD	0.022440
NN with Adam	0.394109
NN with Adam & Dropout	0.465638
NN with SMOTE & SGD	0.722300

	recall
NN with SMOTE & Adam	0.722300
NN with SMOTE,Adam & Dropout	0.768583

train_metric_df - valid_metric_df

	recall
NN with SGD	-0.002805
NN with Adam	-0.007013
NN with Adam & Dropout	0.063114
NN with SMOTE & SGD	-0.015806
NN with SMOTE & Adam	0.171492
NN with SMOTE,Adam & Dropout	0.087175

```
y_test_pred = model_4.predict(X_test)    ## Complete the code to
    specify the best model
y_test_pred = (y_test_pred > 0.4)
print(y_test_pred)
```

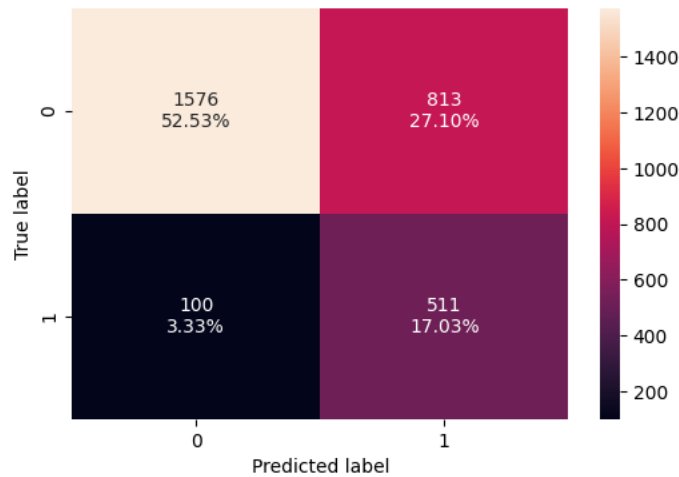
94/94 ————— 0s 3ms/step

```
[[False]
 [ True]
 [False]
 ...
 [ True]
 [ True]
 [False]]
```

```
#lets print classification report
cr=classification_report(y_test,y_test_pred)
print(cr)
```

	precision	recall	f1-score	support
0.0	0.94	0.66	0.78	2389
1.0	0.39	0.84	0.53	611
accuracy			0.70	3000
macro avg	0.66	0.75	0.65	3000
weighted avg	0.83	0.70	0.73	3000

```
#Calculating the confusion matrix
make_confusion_matrix(y_test,y_test_pred)
```



Actionable Insights and Business Recommendations

##1. High Risk Customer Segments

###Geography & Gender:

-If your EDA showed higher churn in specific countries (Germany) or genders (females), target retention campaigns there

Recommendation: Offer localized incentives (lower fees for German customers) or gender specific financial planning services

###Age Groups:

-Older customers (>50 years) tend to churn more

Recommendation: Create loyalty programs for seniors (higher interest rates on deposits)

###Low CreditScore Customers:

-Customers with poor credit history may leave due to rejected loan applications

Recommendation: Provide credit counseling or "credit repair" partnerships

##2. Product & Service Improvements

###NumOfProducts:

-Customers with only 1 product churn more (they're less "sticky")

Recommendation: Bundle products ("Get a free credit card with a savings account")

###Inactive Members:

-Dormant accounts are likely to churn

Recommendation: Reactivation campaigns ("We miss you! Get \$10 for your next transaction")

##3. Model Driven Retention Strategies

###Threshold Tuning:

Best model (model_4 with SMOTE + Adam) achieved 77% recall for churn (class 1) but had lower precision (36%)

Recommendation: Use a lower threshold (0.4) to flag more at risk customers, even if some false positives are included

###Targeted Interventions:

####For predicted churners (high recall), prioritize:

-High Balance + Low Activity: Offer personalized financial advice

-Low Tenure: Assign a dedicated account manager for new customers

##4. Operational Changes

###Customer Support:

-Churners often face unresolved issues

Recommendation: Implement a VIP support line for high risk customers

###Pricing Transparency:

-Hidden fees drive churn

Recommendation: Simplify fee structures and notify customers before charges

##5. Long Term Data & Model Improvements

###Feature Engineering:

-Add interaction terms ($\text{Balance} \times \text{NumOfProducts}$) to capture nuanced behavior

###Regular Retraining:

-Update the model quarterly to adapt to changing customer patterns

###Testing:

Test retention strategies (control groups) to measure ROI.

Power Ahead ____